CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: **Therapea: A Mental Health Triage and Telehealth System**

Prepared By: **Libato, Michael Grant S.**

Version: 3

Date: May 7, 2026
Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1	02/18/2026	Libato, Michael Grant S.	Initial draft of Triage and Telehealth modules	Draft
2	03/03/2026	Libato, Michael Grant S.	Integrated Agora WebRTC architecture, specified Telehealth REST endpoints, and defined video performance NFRs.	Review
3	05/07/2026	Libato, Michael Grant S.	Updated UI frames for Checkout Dashboard, added Registration page mockups, and documented final video call implementation.	Review

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY

1.1 Project Overview

Therapea is a progressive web application (PWA) designed to bridge the gap between mental health awareness and professional intervention. Unlike standard booking apps, Therapea utilizes a "Smart Triage Engine", a rule-based assessment system that evaluates a user's current mental state and routes them to the appropriate level of care: self-help content, teleconsultation, or physical psychiatric intervention.

Built upon a robust Layered Architecture, the system ensures secure, scalable, and maintainable operations. It integrates industry-standard services including Google OAuth for authentication, PayMongo for secure payment processing, and Agora for realtime telehealth, ensuring a seamless and professional user experience.

1.2 Objectives

1. Develop a Smart Triage Algorithm that categorizes user distress levels based on clinical scales (e.g., GAD-7/PHQ-9 logic).
2. Implement secure Authentication & Authorization using Google OAuth and RoleBased Access Control (RBAC) with JWT.
3. Integrate a Payment Gateway (PayMongo) in Test Mode to securely handle consultation fees.
4. Implement real-time Telehealth Video Consultations using the Agora Web SDK.
5. Develop an automated SMTP Email Notification System for account verification and appointment confirmations.
6. Create a Geolocation-based Facility Locator via Google Maps API to identify accredited psychiatric clinics for urgent cases.
7. Ensure compliance with the Data Privacy Act of 2012 through encrypted data storage and strict Row Level Security (RLS).

1.3 Scope

Included Features:

- Smart Triage Engine: Automated assessment of user mental state.
- Telehealth System: Real-time video calls with verified psychologists/psychiatrists.

- Secure Authentication: Google OAuth login and JWT-based session management.
- Payment Integration: Functional payment processing using PayMongo/Xendit (Sandbox/Test Mode).
- Notifications: Automated email alerts for bookings and registration.
- File Management: Secure upload and storage of Doctor PRC IDs for verification.
- Geolocation: Google Maps integration to display nearby physical clinics.
- Content Library: Repository of mental health articles and studies.
- User Roles: Patient, Doctor (Verified), Administrator.

Excluded Features:

- Generative AI Diagnosis: The system uses deterministic rule-based logic, not LLMs, to prevent hallucinated medical advice.
- Digital Prescriptions: Generation of e-prescriptions is out of scope due to complex legal requirements.
- Live Production Payments: The system will process transactions in a Test/Sandbox Environment only; no real currency will be deducted.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the technical blueprint for Therapea. It details the implementation of the "Stepped Care" model within a secure Layered Architecture. The document specifies how the system integrates mandatory enterprise features, including Google OAuth, Payment Gateways, and SMTP Notifications, to ensure a professional, secure, and scalable mental health platform.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

- **Project Name:** Therapea
- **Domain:** HealthTech / Mental Wellness
- **Problem Statement:** Individuals often delay seeking help or seek the wrong type of help due to lack of guidance and accessible tools.
- **Solution:** A triage-first system that guides users to the correct resource automatically, supported by secure telehealth and payment integrations.

2.2 Core User Journeys

Journey 1: The "Low Risk" User (Self-Care & OAuth)

1. User logs in using Google OAuth.
2. User completes the "Daily Check-in" (Triage Form).
3. System calculates a "Low Risk" score.
4. System recommends self-paced breathing exercises and articles.

Journey 2: The "Medium Risk" User (Booking & Payment)

1. User completes Triage; System detects "Moderate Anxiety."
2. User selects a Doctor and a time slot.
3. System prompts for Payment: User enters details in the PayMongo/Xendit (Test) gateway.
4. Upon success, system confirms booking and sends an Email Notification with the video link.
5. At the scheduled time, User joins the secure video room.

Journey 3: The "High Risk" User (Crisis Locator)

1. User completes Triage; System detects "Severe Distress."
2. System displays a Crisis Alert and activates the Geolocation Locator.
3. Map displays the 3 nearest psychiatric facilities.
4. "One-Tap Call" connects user to the National Center for Mental Health (NCMH).

Journey 4: Doctor Verification (Admin & Email)

1. Doctor uploads PRC ID file (File Upload).
2. Admin reviews the file in the dashboard.
3. Admin clicks "Verify."
4. System updates status and sends an "Account Approved" Email to the doctor.

2.3 Feature List (MoSCoW) MUST

HAVE:

- Google OAuth Login (Req 4.2).

- Smart Triage Logic Engine.
- Payment Gateway Integration (Test Mode) (Req 4.4).
- SMTP Email Notifications (Req 4.6).
- File Upload for Doctor IDs (Req 4.3).
- Video Consultation (Agora).

SHOULD HAVE:

- Geolocation for clinics (Req 4.1).
- User Dashboard with History.

COULD HAVE:

- Mood Tracker Graph.

WON'T HAVE:

- AI Chatbot (LLM).
- Live Real-Money Transactions.

2.4 Detailed Feature Specifications Feature:

Authentication & Security (Req 4.2)

- Screens: Login, Register.
- Fields: Email, Password, OAuth Provider.
- Validation: Bcrypt password hashing, JWT token generation. ● Process:
    - Google Login: System authenticates via Google, checks DB for user, creates session.
    - Standard Login: Verifies hash, issues JWT.
- API Endpoints: POST /auth/google, POST /auth/login, GET /auth/me.

Feature: Smart Triage Engine

- Screens: Triage Chat.
- Fields: 10 Standardized Questions (Likert 1-4).

- Logic: Summation of scores -> Categorization (Low/Medium/High) -> Routing. ● API Endpoints: POST /triage/submit, GET /triage/history.

Feature: Appointment & Payment Integration (Req 4.4)

- Screens: Booking Calendar, Checkout Modal.
- Functions: Select Slot -> Create Payment Intent -> Validate Payment -> Book Slot.
- Integration: PayMongo/Xendit API (Test Mode).
- Process: Appointment is only "Confirmed" after the payment webhook returns "Paid."
- API Endpoints: POST /payments/create-intent, POST /payments/webhook.

Feature: Email Notification Service (Req 4.6)

- Trigger: Successful Booking OR Account Verification.
- Integration: SMTP (e.g., Resend, Nodemailer).
- Output: HTML Email sent to user's registered address.
- API Endpoints: Internal Service Call (not exposed to client).

Feature: File Upload & Verification (Req 4.3)

- Screens: Doctor Registration.
- Fields: File Input (Image/PDF).
- Process: Upload to Cloud Storage -> Save URL in DB -> Admin Verification.
- API Endpoints: POST /doctor/upload-license.

Feature: Telehealth Video Consultation

- Screens: Video Room.
- Integration: Agora Web SDK.
- Security: Token generation required; room accessible only during slot time. ● API Endpoints: GET /video/token.

Feature: Geolocation Facility Locator (Req 4.1)

- Screens: Map View.
- Integration: Google Maps API.
- Process: Convert User Lat/Long -> Query Places API -> Display Markers. ● API Endpoints: GET /clinics/nearby.

2.5 Acceptance Criteria

AC-1: Google OAuth Login

Given I am on the login page

When I click "Continue with Google"

Then I should be redirected to Google and back to the dashboard

And a valid JWT session should be created.

AC-2: Successful Booking with Payment

Given I have selected a doctor and time slot

When I complete the payment using Test Credentials

Then the system should display "Payment Successful" And the

appointment status should change to "Confirmed"

And I should receive a confirmation email.

AC-3: Admin Doctor Verification

Given I am an Admin viewing a pending doctor

When I verify their uploaded ID file and click "Approve"

Then the doctor's status should update to "Verified"

And an approval email should be sent to the doctor.

AC-4: High Risk Triage Flow

Given I answer the triage questions with a high score (>20)

Then the system should immediately route me to the Geolocation Map

And display the Crisis Hotline button.

AC-5: File Upload

Given I am registering as a Doctor

When I upload a valid JPG/PNG of my license

Then the file should be stored on the server

And the file URL should be linked to my profile in the database.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API Response Time: REST API endpoints (Controller Layer) must respond within ≤ 2 seconds for standard CRUD operations.
- Real-Time Latency: WebSocket notifications (for Appointment Updates) must be delivered within ≤ 500ms.
- Payment Processing: Payment Intent creation via PayMongo/Xendit (Test Mode) must complete within ≤ 5 seconds.
- Video Initialization: Agora video calls must connect (handshake) within 5 seconds on 4G/LTE networks.
- Concurrency: System must support at least 20 simultaneous active users (including 5 concurrent video sessions).
- Cold Start: Initial PWA load time must be ≤ 4 seconds, with subsequent navigation under 1 second (Single Page Application).

3.2 Security Requirements

- Encryption: HTTPS (TLS 1.2+) is strictly enforced for all client-server communication.
- Authentication:
  - Google OAuth: Must validate the provider token and exchange it for an internal JWT (JSON Web Token).
  - Session Control: JWTs have a 1-hour expiration time; refresh tokens are used to maintain session validity securely.
- Password Security: All user passwords (for non-OAuth accounts) are hashed using BCrypt with a minimum of 10 salt rounds.
- Access Control: Role-Based Access Control (RBAC) is enforced at the API Middleware Level.
  - Example: Only users with role: 'admin' can access /api/admin/* endpoints.
- Data Privacy:
  - Video/Audio: Streams are Peer-to-Peer (P2P) and are never recorded or stored on the server.

    ○ Medical Data: Row Level Security (RLS) ensures patients can only query their own assessment records.

3.3 Compatibility Requirements

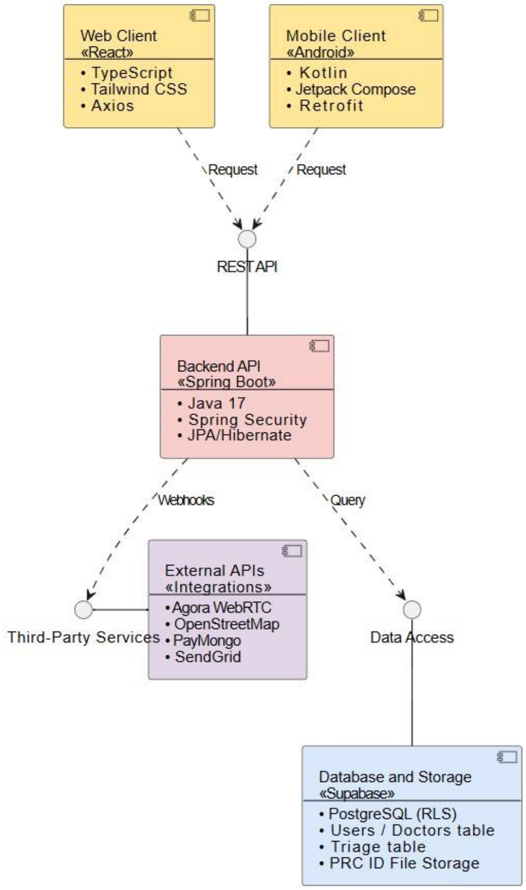
- Mobile OS: Android 8.0 (Oreo) and above; iOS 14 and above.
- Browsers: Chrome (v90+), Safari (v14+), Edge (Current). Note: Internet Explorer is not supported.
- Network: Functional on unstable 3G/4G connections. The UI must display a "Reconnecting..." toast notification if the WebSocket disconnects.
- Device Hardware: Requires active permissions for Camera, Microphone (Telehealth), and GPS (Clinic Locator).

3.4 Usability Requirements

- Cognitive Load: The "Crisis Mode" (Hotlines/Map) must be accessible within 2 clicks from the dashboard.
- Error Handling: System errors must be caught globally and displayed in plain English (e.g., "Payment Failed" instead of "Error 500").
- Accessibility: Text contrast ratio of at least 4.5:1 (WCAG AA).
- Touch Targets: Interactive elements (Buttons/Links) must be at least 48x48 pixels on mobile viewports.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot, Spring Security (JWT), Spring Data JPA / Hibernate
- **Database & Storage:** Supabase (PostgreSQL 14+ with Row Level Security & Supabase Storage for PRC IDs)
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Android (Kotlin), XML-Based UI Layouts (API Level 34 / Android 14), Retrofit (API Client)
- **External APIs (Integrations):**
  - Telehealth Video: Agora WebRTC SDK ○ Geolocation & Maps: OpenStreetMap API ○ Payments: PayMongo API ○ Email Notifications: SendGrid SMTP API
- **Build Tools:** Maven (Backend), npm/yarn (Web Frontend), Gradle (Android)
- **Deployment:** Railway/Render (Backend API), Vercel (Web Frontend), APK Distribution (Mobile)

Architecture Pattern Explanation: The backend strictly follows a Layered Architecture pattern to cleanly separate concerns, ensure scalability, and maintain high security standards:

- **Controller Layer:** Acts as the entry point for the application. It handles incoming HTTP REST requests, parses incoming payloads, and routes them to the appropriate services.
- **Service Layer:** Contains the core business rules of the platform (e.g., executing the Smart Triage scoring logic, validating PayMongo webhooks, and managing doctor availability).
- **Repository Layer:** Utilizes Spring Data JPA interfaces to safely execute database queries and perform CRUD operations on the PostgreSQL database.
- **DTOs**: Used exclusively to transfer data between the client and server. This strictly ensures that sensitive database entities (like user passwords and internal IDs) are never exposed in API responses.
- **Global Exception Handling:** Implemented globally via @ControllerAdvice to catch system errors and format them into standardized, predictable HTTP JSON error responses rather than crashing the application.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://api.therapea.com/api/v1
Format	application/json for standard payloads; multipart/form-data for file uploads (e.g., PRC IDs).
Authentication	Stateless JWT (JSON Web Token) passed via the Authorization: Bearer <token> header.
Pagination	Standardized via **?page=0&size=20** query parameters for list endpoints.
Standard Response Envelope	All successful responses wrap the payload in a standard structure to ensure predictable client-side parsing.

Response Structure:	{
	"success": true,
	"data": { ... },
	"meta": {
	"timestamp": "2026-0301T22:30:00Z",
	"path": "/api/v1/triage/assess"
	}
	}

5.2 Endpoint Specifications

Authentication Endpoints

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"id": <id>,
	"email": <email>,
	"fullName": <fullName>,
	"role": <role>
	},
	"accessToken": <token>,
	"refreshToken": <token>
	},
	"error": {
	"code": string,
	"message": string,

	"details": object }

Description	User Logout
API URL	/auth/logout
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None
Response Structure	

Description	User Login
API URL	/auth/login
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>
	
	}

Response Structure	

Description	User Login
API URL	/auth/login
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>
	
	}

Response Structure	{
	"success": boolean,
	
	"data": {
	
	"message": "Logged out successfully"
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Refresh Token
API URL	/auth/refresh
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"refreshToken": <refreshToken>
	
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"token": <token>,
	
	"refreshToken": <token>
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

User Profile Endpoints

Description	Get User Profile
API URL	/users/me
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token
Request Payload	None

Response Structure	{
	"success": boolean,
	
	"data": {

	"user": {
	"id": <id>,
	"email": <email>,
	"fullName": <fullName>,
	"role": <role>,
	"phone": <phone>,
	"createdAt": <createdAt>
	}
	},
	
	"error": {
	"code": string,
	"message": string,
	"details": object
	}

Description	Update User Profile
API URL	/users/me
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token

Request Payload	{
	
	"fullName": <fullName?>,
	
	"phone": <phone?>,
	
	"emergencyContact": <emergencyContact?>
	}

Response Structure	{
	"success": boolean,
	
	"data": {
	
	"user": { ...updated user fields... }
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Doctor & Verification Endpoints

Description	Upload PRC ID
API URL	/doctors/upload-prc
HTTP Request Method	POST
Format	multipart/form-data (Request) / JSON
Authentication	Bearer Token (Authenticated - Doctor only)

Request Payload	{
	
	"prcImage": <file>
	
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"message": "PRC ID uploaded successfully",
	
	"status": "PENDING"

	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Get Doctors List
API URL	/doctors
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	Query Parameters: ?page=1&limit=20&status= APPROVED&search=keyword

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"doctors": [...doctor objects...],
	
	"pagination": {
	
	"page": <page>,
	
	"limit": <limit>,
	
	"total": <total>,
	
	"pages": <pages>
	
	}
	
	},

	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	
	}

Description	Get Doctor Details
API URL	/doctors/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{ "success": boolean, "data": { "doctor": { "id": <id>, "fullName": <fullName>, "prcStatus": <prcStatus>, "biography": <biography>, "hourlyRate": <hourlyRate> } }, "error": { "code": string, "message": string, "details": object
	}

Description	Update Doctor Status
API URL	/admin/doctors/{id}/status
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token (Admin only)
Request Payload	{ "status": "APPROVED" }

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"message": "Doctor status updated",
	
	"doctor": { ...updated doctor details... }
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Update Doctor Status
API URL	/admin/doctors/{id}/status
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token (Admin only)

Request Payload	{
	
	"status": "APPROVED"
	}

Response Structure	{ "success": boolean,

	"data": {
	
	"message": "Doctor status updated",
	
	"doctor": { ...updated doctor details... }
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Smart Triage Endpoints

Description	Get Triage Questions
API URL	/triage/questions
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"questions": [
	
	{
	
	"id": <id>,
	
	"text": <text>,
	
	"options": [
	
	{ "score": <score>, "label": <label> }

	]

	}
	
	]
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Submit Triage Answers
API URL	/triage/submit
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Patient only)

Request Payload	{
	"answers": [
	
	{ "questionId": <questionId>, "score": <score> }
	
	]
	
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"assessment": {
	
	"id": <id>,
	
	"totalScore": <totalScore>,
	
	"riskLevel": <riskLevel>,

	"recommendedAction": <recommendedAction>,
	
	"date": <date>
	
	}
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Get Triage History

API URL	/triage/history
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Authenticated)
Request Payload	None

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"assessments": [...assessment objects...
	]
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Get Triage Details
API URL	/triage/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Authenticated)
Request Payload	None

Response Structure	{
	
	 "success": boolean,
	
	 "data": {
	
	 "assessment": { ...assessment details... }
	
	 },
	
	 "error": {
	
	 "code": string,
	
	 "message": string,
	
	 "details": object
	 }

Appointment Endpoints

Description	Get Available Appointments
API URL	/appointments/available
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	Query Parameters: ?doctorId=uuid&date=YYYY-MM-DD

Response Structure	{
	
	 "success": boolean,
	
	 "data": {
	
	 "availableSlots": ["09:00", "10:00", "13:00", "15:00"]

	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Book Appointment
API URL	/appointments
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Patient only)

Request Payload	{
	
	"doctorId": <doctorId>,
	
	"scheduledDate": <scheduledDate>,
	
	"scheduledTime": <scheduledTime>
	
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"appointment": {
	
	"id": <id>,
	
	"doctorId": <doctorId>,
	
	"patientId": <patientId>,
	
	"status": "PENDING_PAYMENT",

	"scheduledDate": <scheduledDate>
	
	}
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Get Appointments
API URL	/appointments
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Authenticated)
Request Payload	Query Parameters: ?status=CONFIRMED&role=doctor

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"appointments": [...appointment objects...],
	
	"pagination": { "page": <page>, "limit": <limit> }
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object

	}

Description	Get Appointment Details
API URL	/appointments/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Authenticated)
Request Payload	None

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"appointment": { ...appointment details... }
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Update Appointment Status
API URL	/appointments/{id}/status
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token (Authenticated)

Request Payload	{
	"status": "CANCELLED"
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"message": "Appointment updated",

	"appointment": { ...updated details... }
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Payment Endpoint

Description	Create Payment Intent
API URL	/payments/create-intent
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Authenticated)

Request Payload	{
	
	"appointmentId": <appointmentId>,
	
	"amount": <amount>
	
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"checkoutUrl": <checkoutUrl>,
	
	"paymentIntentId": <paymentIntentId>,
	
	"status": "REQUIRES_PAYMENT"
	
	},

	"error": {
	"code": string, "message": string, "details": object }

Description	Payment Webhook
API URL	/payments/webhook
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	
	"data": {
	
	"attributes": {
	
	"type": <type>,
	
	"data": {
	
	"attributes": {
	
	"status": <status>,
	
	"amount": <amount>
	
	}
	
	}
	
	}
	
	}
	
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"message": "Webhook processed"

	}, "error": { "code": string, "message": string, "details": object

	}

Telehealth & Geolocation Endpoints

Description	Get Telehealth Token
API URL	/telehealth/token
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Participants only)
Request Payload	Query Parameters: ?appointmentId=uuid

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"agoraToken": <agoraToken>,
	
	"channelName": <channelName>,
	
	"uid": <uid>
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	End Telehealth Session
API URL	/telehealth/end
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Authenticated)

Request Payload	{
	
	"appointmentId": <appointmentId>,
	
	"durationMinutes": <durationMinutes>
	
	}

Response Structure	{
	
	"success": boolean,
	
	"data": {
	
	"message": "Session ended successfully"
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

Description	Get Nearby Facilities
API URL	/facilities/nearby
HTTP Request Method	GET
Format	None
Authentication	Bearer Token (Authenticated)
Request Payload	Query Parameters: ?lat=14.5995&lng=120.9842&radius=5000

Response Structure	{

	"success": boolean,
	
	"data": {
	
	"facilities": [
	
	{
	
	"id": <id>,
	
	"name": <name>,
	
	"address": <address>,
	
	"latitude": <latitude>,
	
	"longitude": <longitude>,
	
	"contactNumber": <contactNumber>,
	
	"distance": <distance>
	
	}
	
	]
	
	},
	
	"error": {
	
	"code": string,
	
	"message": string,
	
	"details": object
	}

5.3 Error Handling

HTTP Status Codes

- **200 OK - Successful request**
- **201 Created -** Resource created (e.g., successfully booked appointment)
- **400 Bad Request -** Invalid input (e.g., missing triage answers or invalid file format)
- **401 Unauthorized -** Authentication required/failed (e.g., missing or expired JWT)
- **403 Forbidden -** Insufficient permissions (e.g., Patient attempting to access Doctor-only endpoints)
- **404 Not Found -** Resource doesn't exist (e.g., Appointment or Doctor profile not found)
- **409 Conflict -** Duplicate resource (e.g., Double-booking an already taken time slot)
- **500 Internal Server Error -** Server error or third-party integration failure (e.g., Agora or PayMongo timeout)

Error Code Examples

Example 1	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials", "details": "Email or password is incorrect"
	},
	"timestamp": "2026-03-01T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "FILE-001",
	"message": "Validation failed",
	"details": {
	"prcImage": "File size exceeds 5MB limit. Please upload a compressed JPG or PNG."
	}
	},

	"timestamp": "2026-03-01T10:30:00Z" }

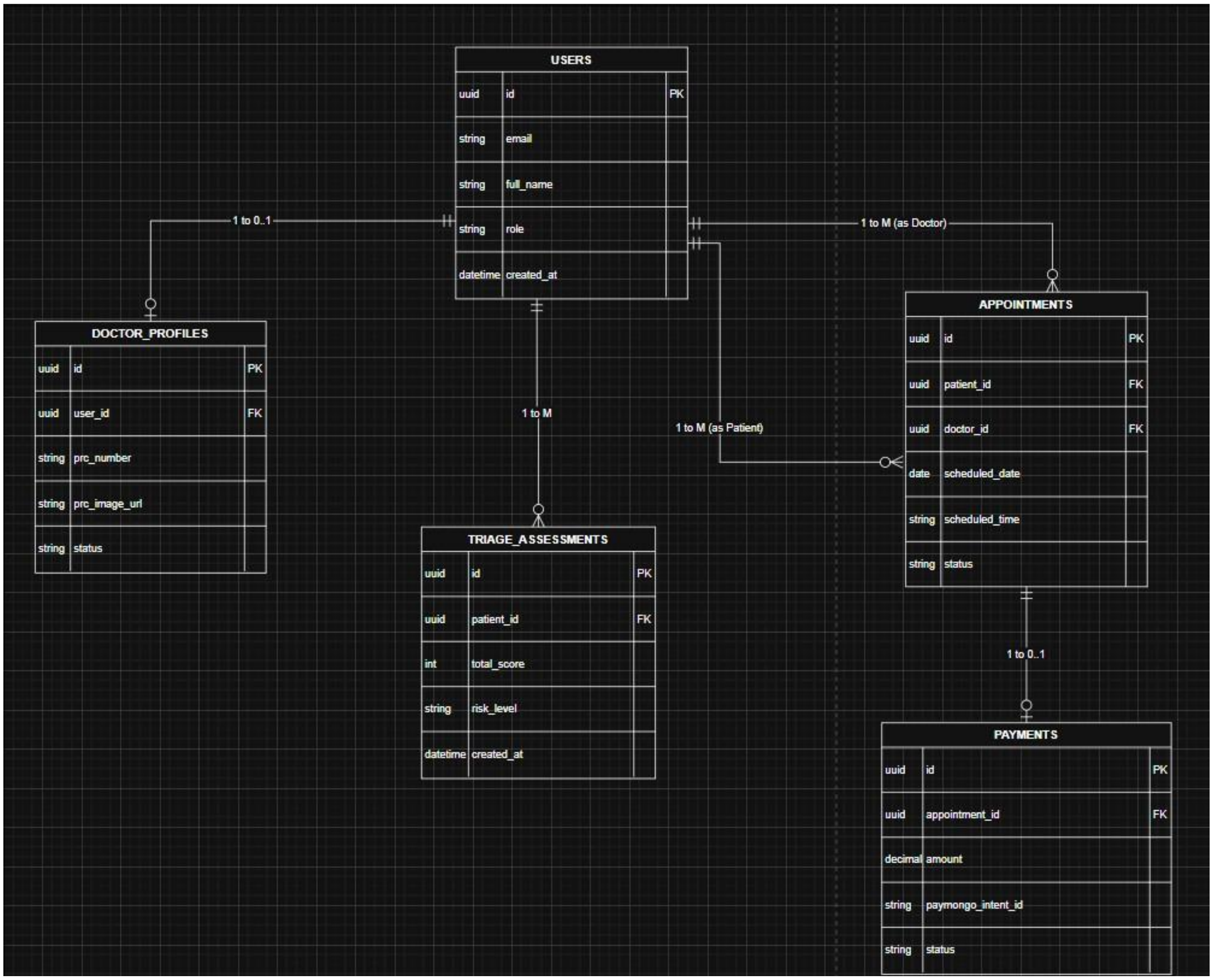
Example 3	{
	 "success": false,
	 "data": null,
	 "error": {
	  "code": "APPT-001",
	  "message": "Scheduling conflict",
	"details": "The requested 2:00 PM time slot has already been booked."
	 },
	 "timestamp": "2026-03-01T10:30:00Z"
	}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions (Role mismatch)
- VALID-001: Validation failed (Missing/Incorrect formatted fields)
- DB-001: Resource not found
- DB-002: Duplicate entry (e.g., Email already registered)
- FILE-001: Invalid file upload (PRC ID size or format error)
- TRIAGE-001: Incomplete triage assessment
- APPT-001: Time slot unavailable or already booked
- PAY-001: Payment checkout creation failed (PayMongo connection error)
- TELE-001: Telehealth room generation failed (Agora App ID/Certificate error) •
  SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Zero-or-One:** User ↔ Doctor_Profiles (A user has exactly one profile if they are a doctor, or zero if they are a patient/admin)
- **One-to-Many:** User (as Patient) → Triage_Assessments (A patient can take multiple triage assessments over time)
- **One-to-Many:** User (as Patient) → Appointments (A patient can book multiple therapy sessions)
- **One-to-Many:** User (as Doctor) → Appointments (A doctor can host multiple therapy sessions)
- **One-to-Zero-or-One:** Appointment ↔ Payments (Each booked appointment generates one specific payment transaction)

Key Tables:

1. **users** - Core user accounts, role identification (Patient, Doctor, Admin), and authentication details.
2. **doctor_profiles** - Verification data, status, and PRC ID file storage links for mental health professionals.
3. **triage_assessments** - Historical records of patient risk scores, serving as the data for the Smart Triage Engine.
4. **appointments** - Scheduling records that act as the central link connecting patients to doctors.
5. **payments** - Financial transaction logs mapped directly to PayMongo checkout sessions.

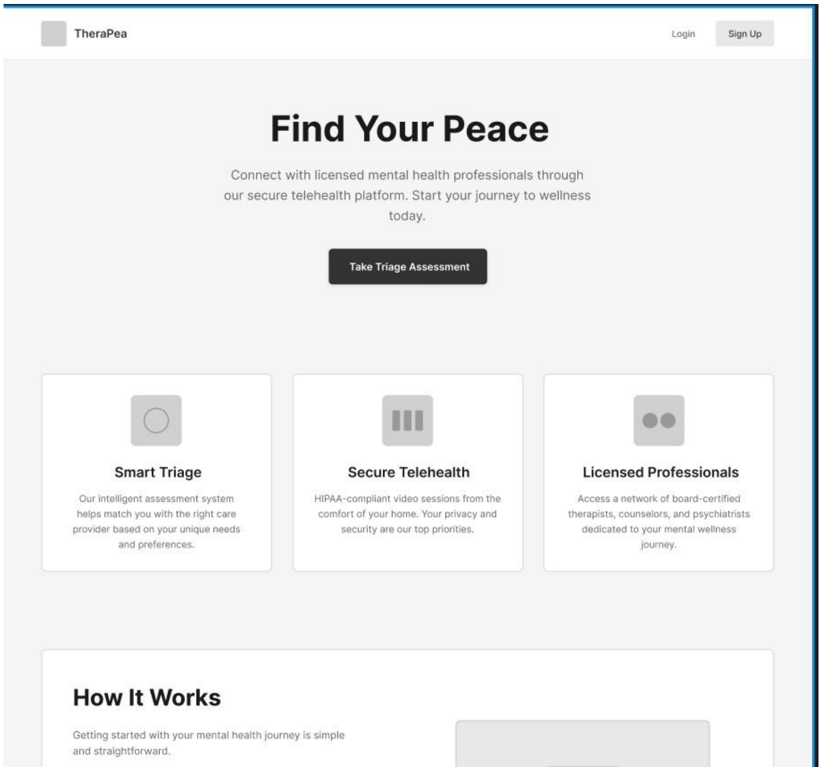
Table Structure Summary:

- **users:** id, email, full_name, role, created_at
- **doctor_profiles:** id, user_id, prc_number, prc_image_url, status
- **triage_assessments:** id, patient_id, total_score, risk_level, created_at
- **appointments:** id, patient_id, doctor_id, scheduled_date, scheduled_time, status
- **payments:** id, appointment_id, amount, paymongo_intent_id, status

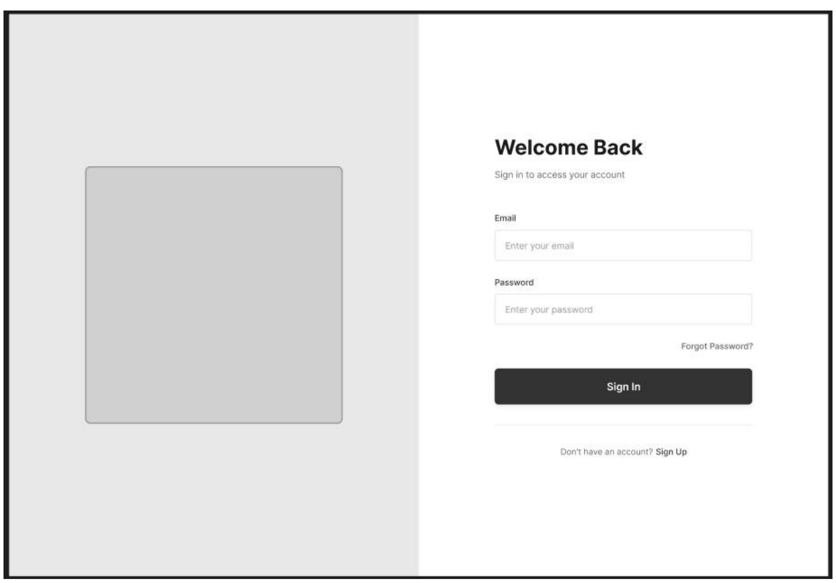
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

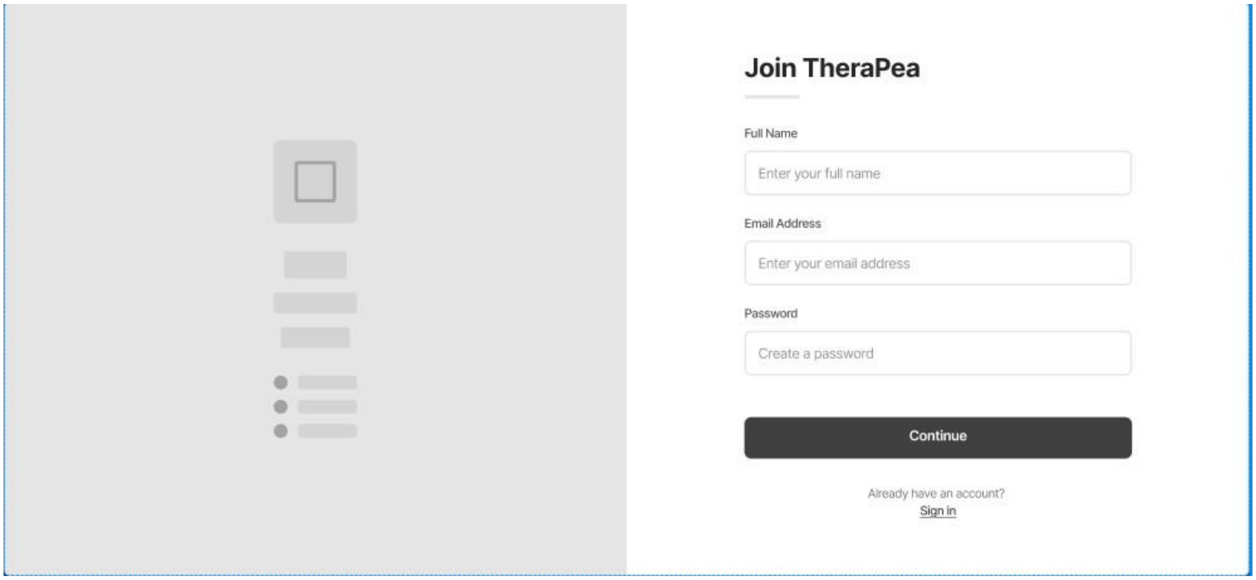
Landing Page:

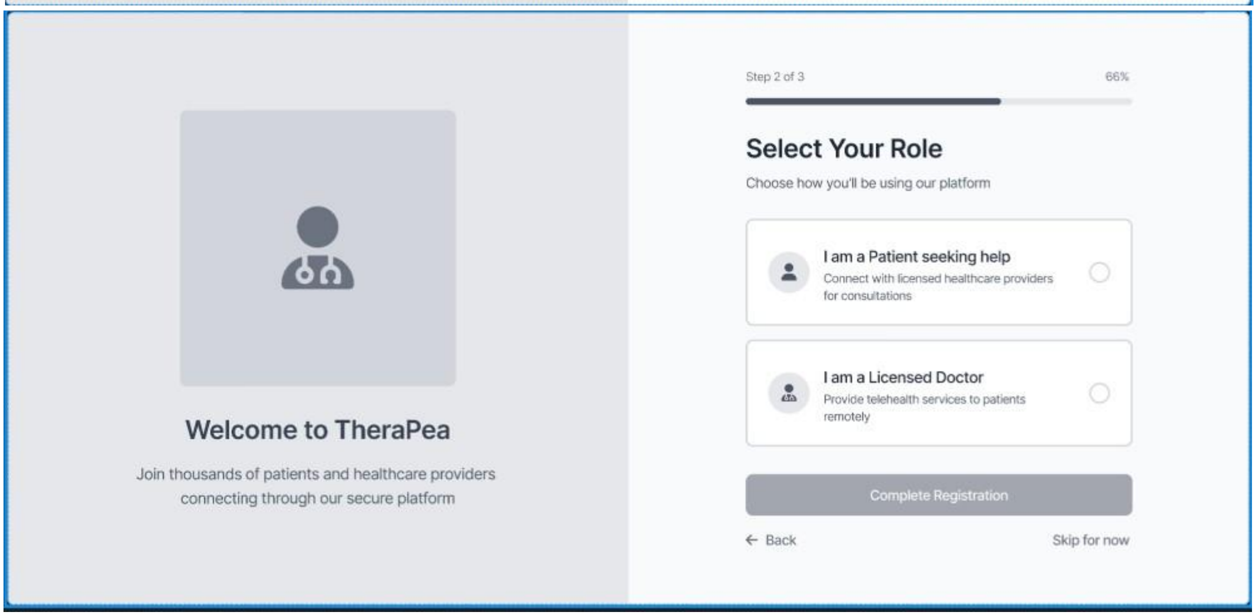


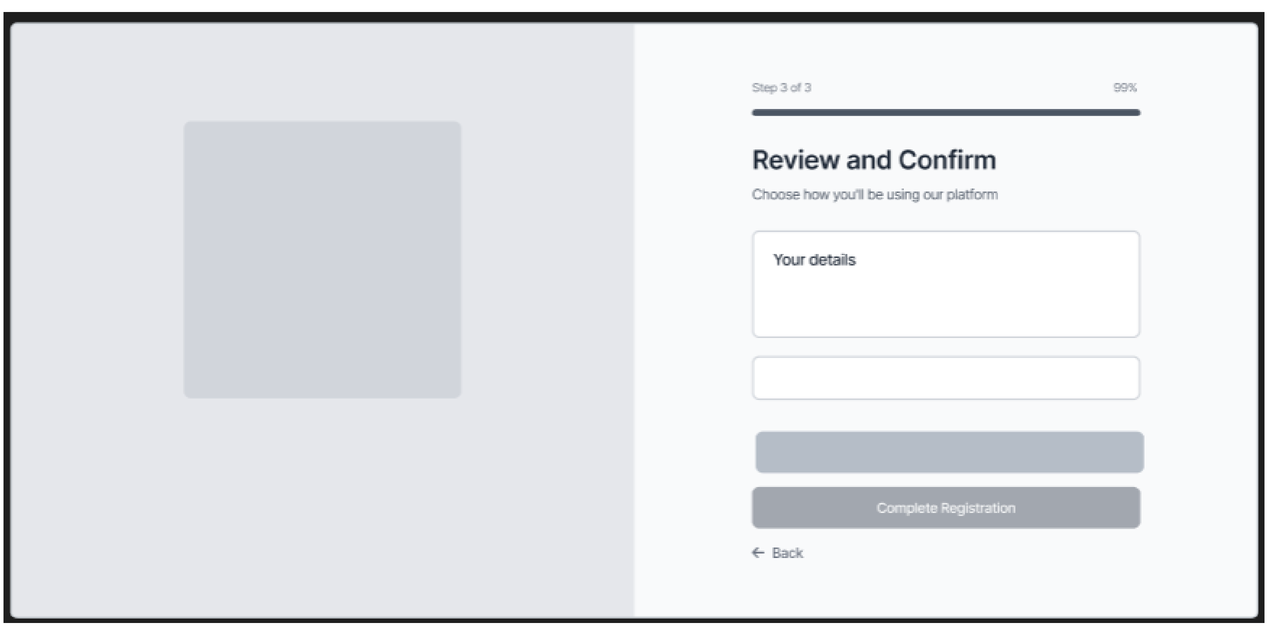
Login Page:



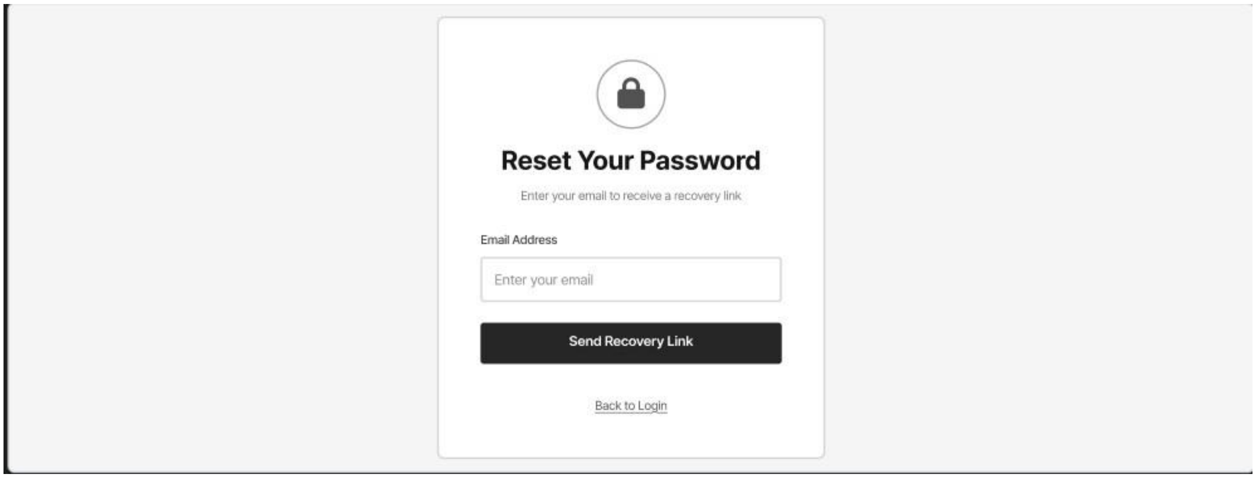
Register Page:


Join TheraPea
Full Name
Enter your full name
Email Address
Enter your email address
Password
Create a password
Continue
Already have an account?
Sign in

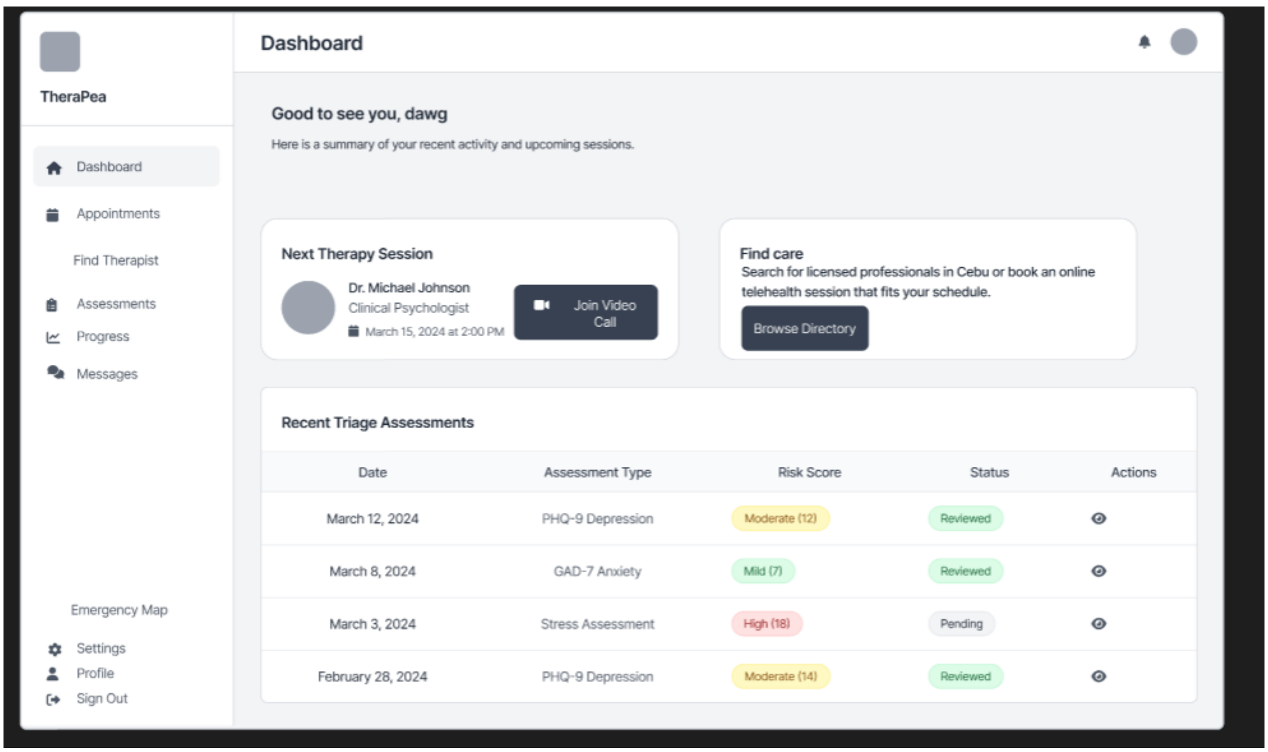

Welcome to TheraPea
Join thousands of patients and healthcare providers connecting through our secure platform
Step 2 of 3
66%
Select Your Role
Choose how you'll be using our platform
I am a Patient seeking help
Connect with licensed healthcare providers for consultations
I am a Licensed Doctor
Provide telehealth services to patients remotely
Complete Registration
Back
Skip for now


Step 3 of 3
99%
Review and Confirm
Choose how you'll be using our platform
Your details
Complete Registration
Back

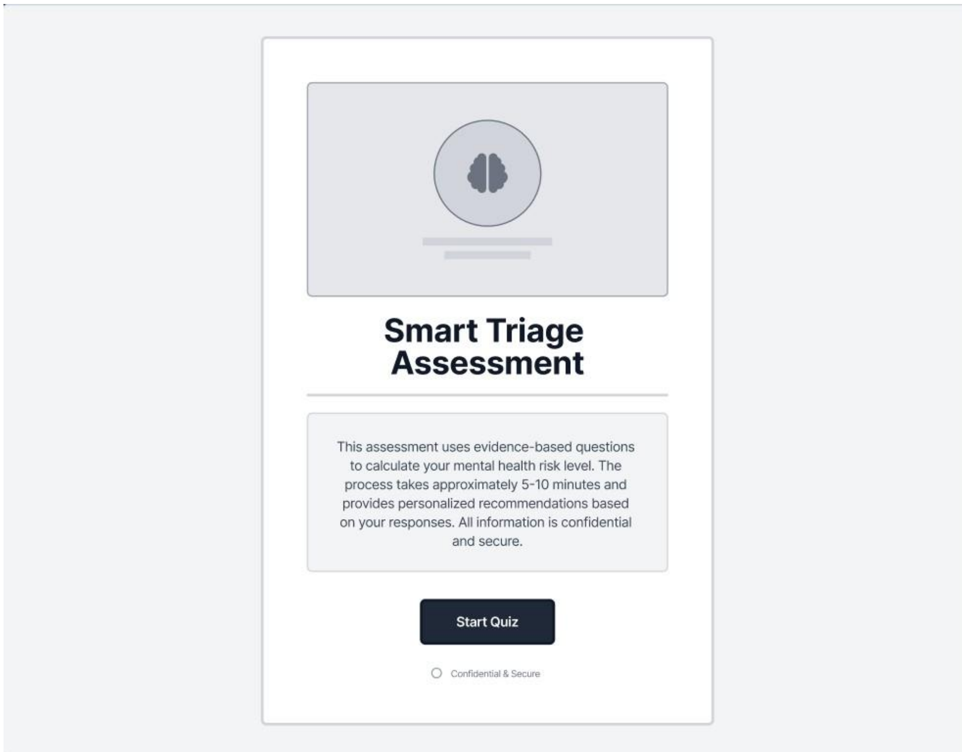
Forgot Password:

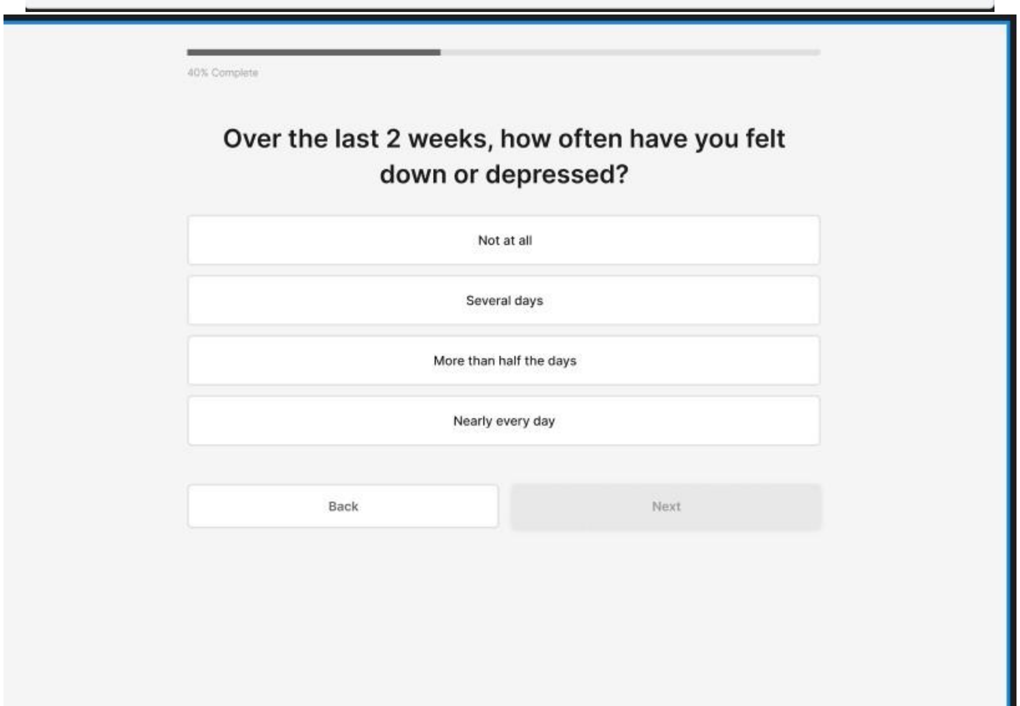


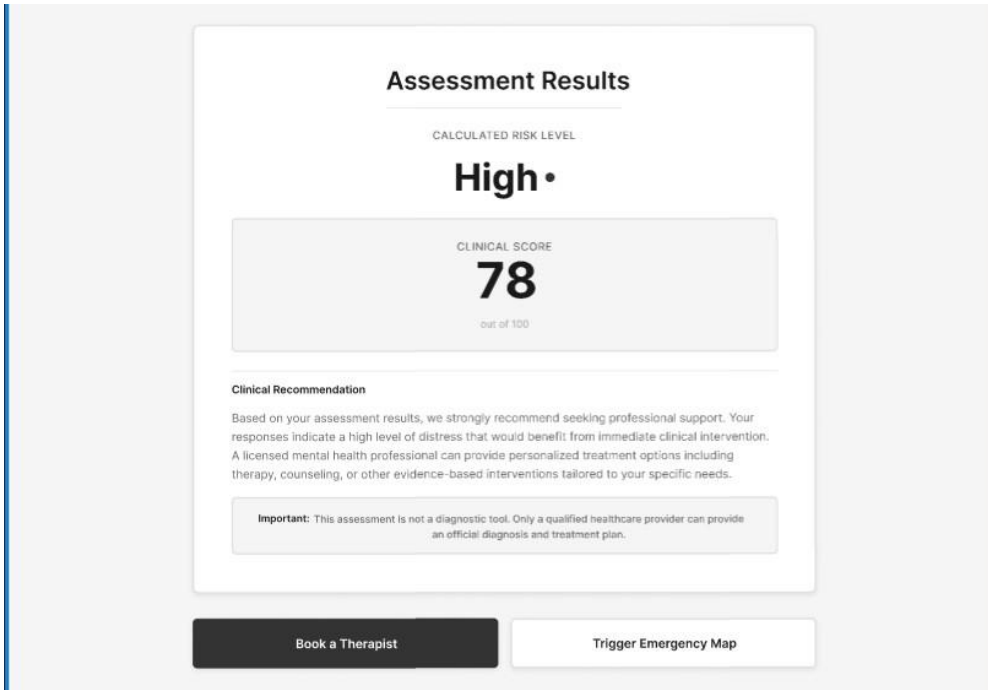
Patient Dashboard:



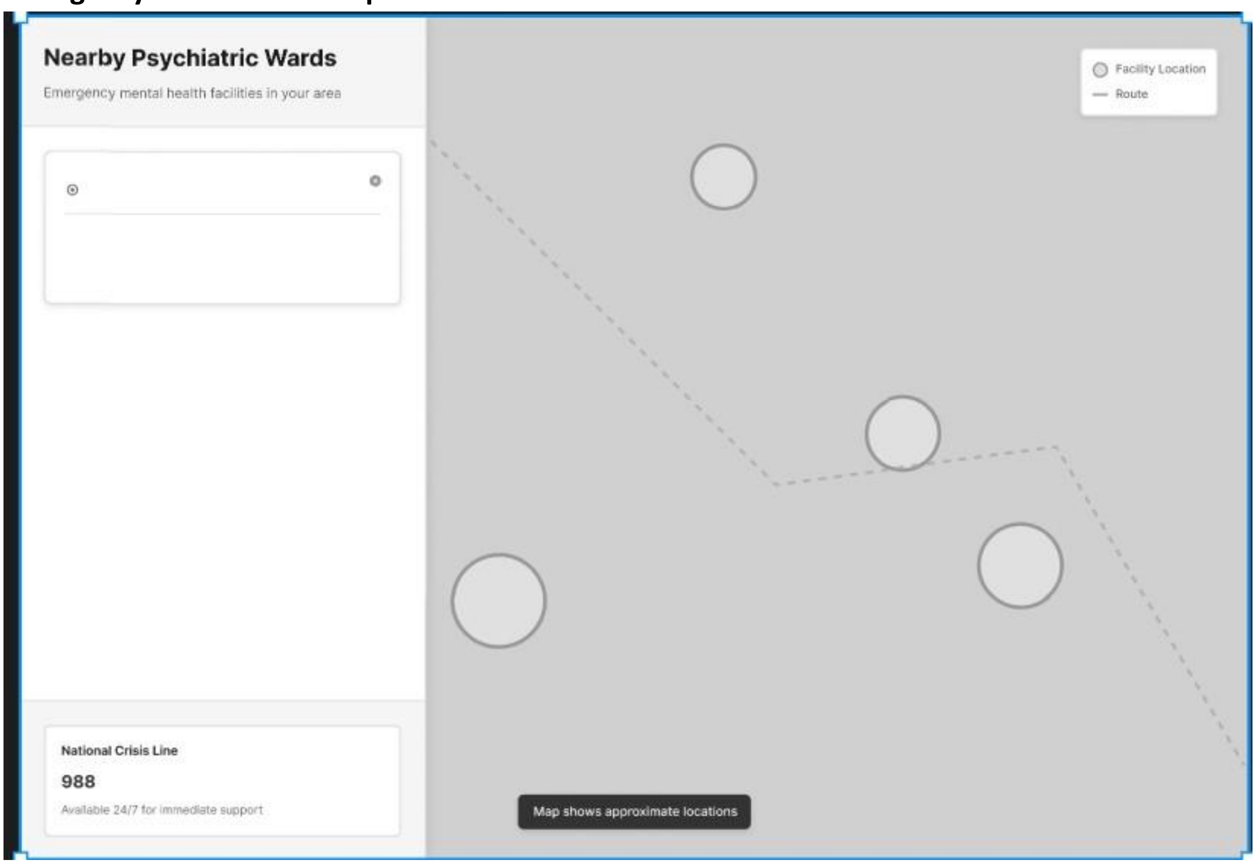
Smart Triage



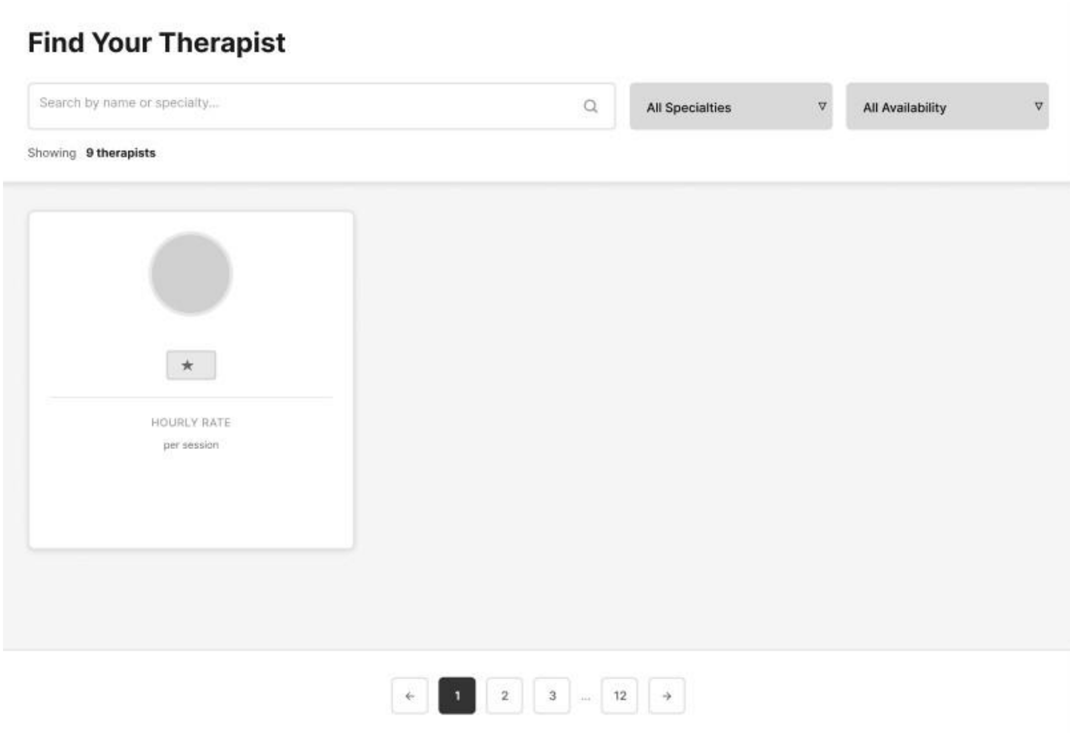




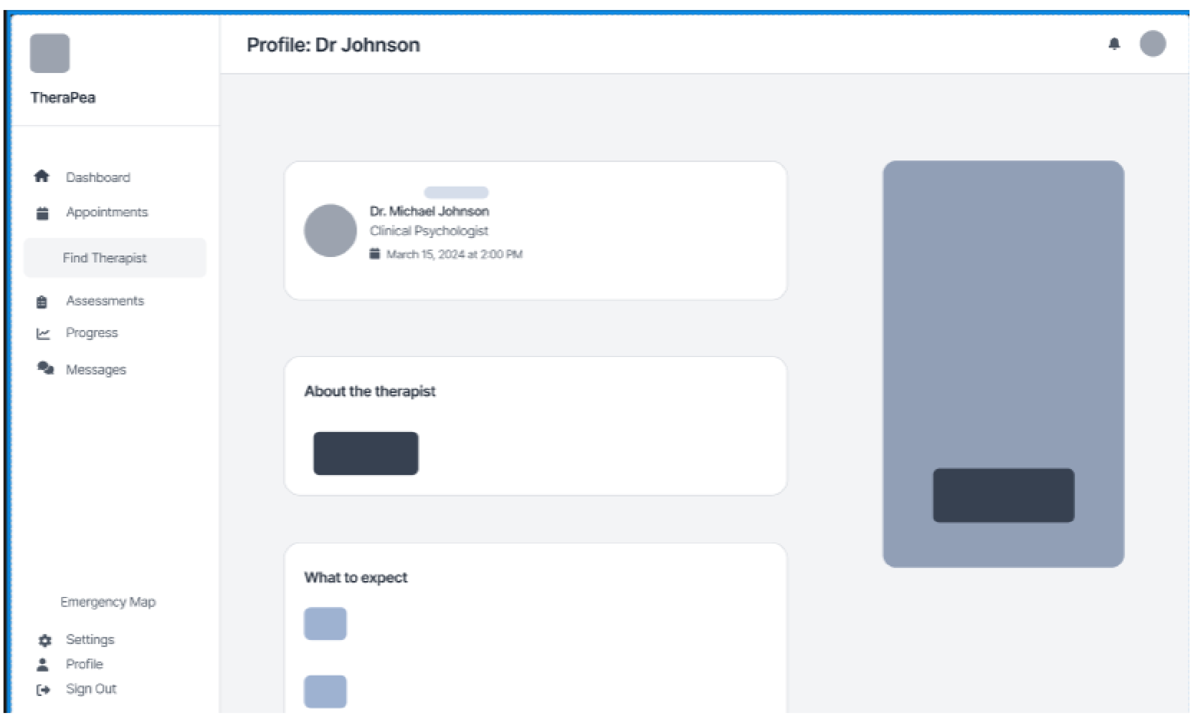
Emergency Geolocation Map:



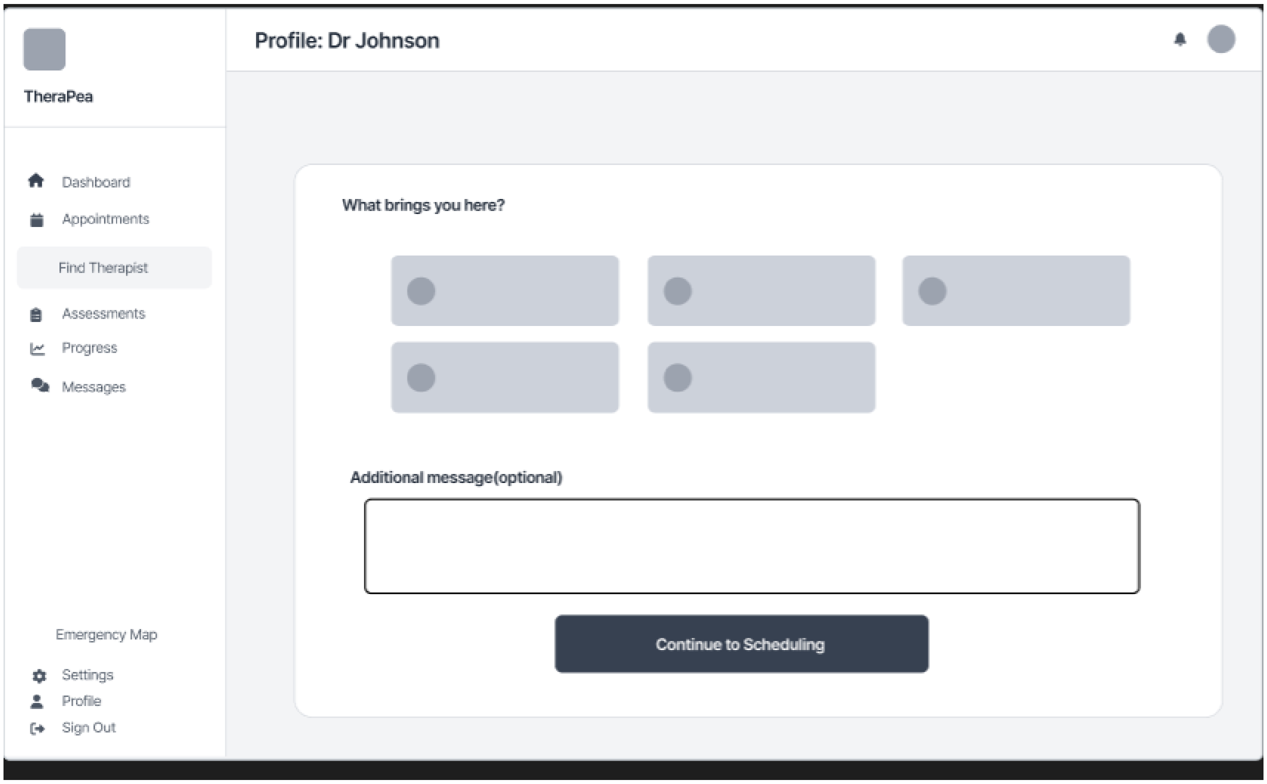
Doctor Directory:

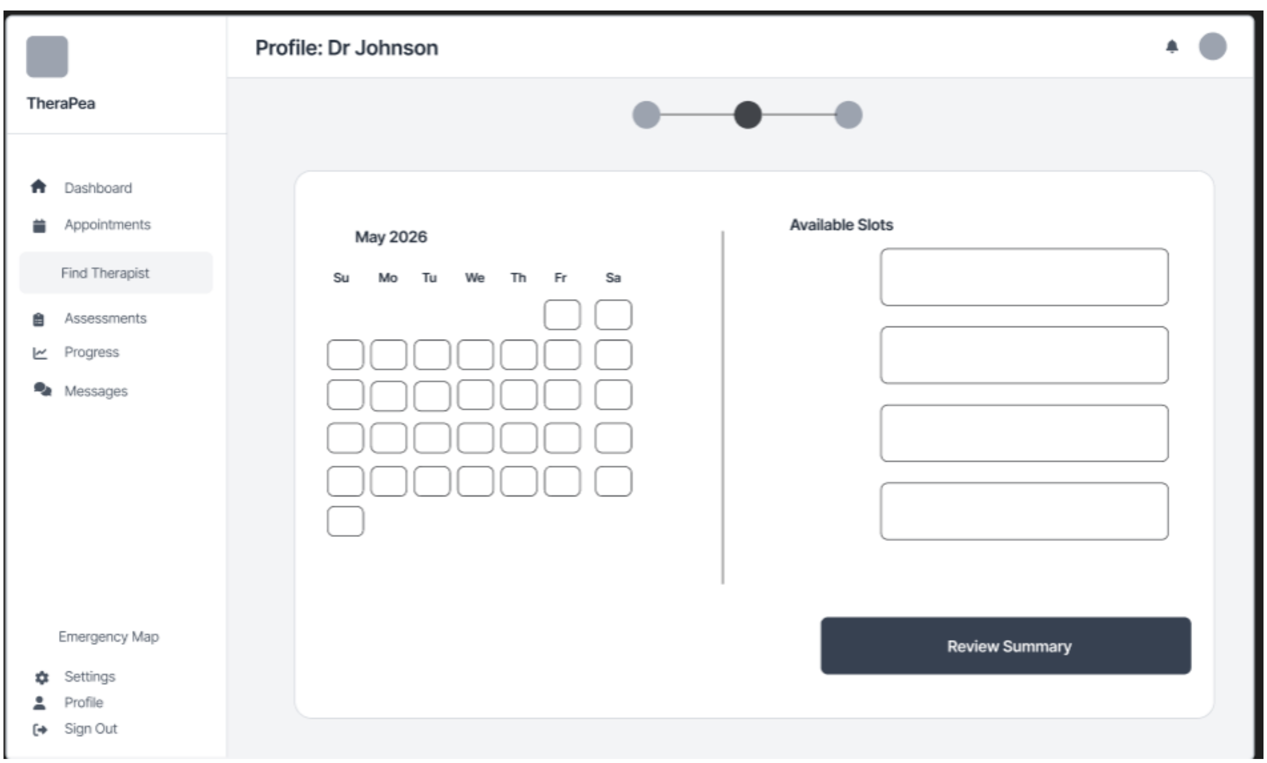


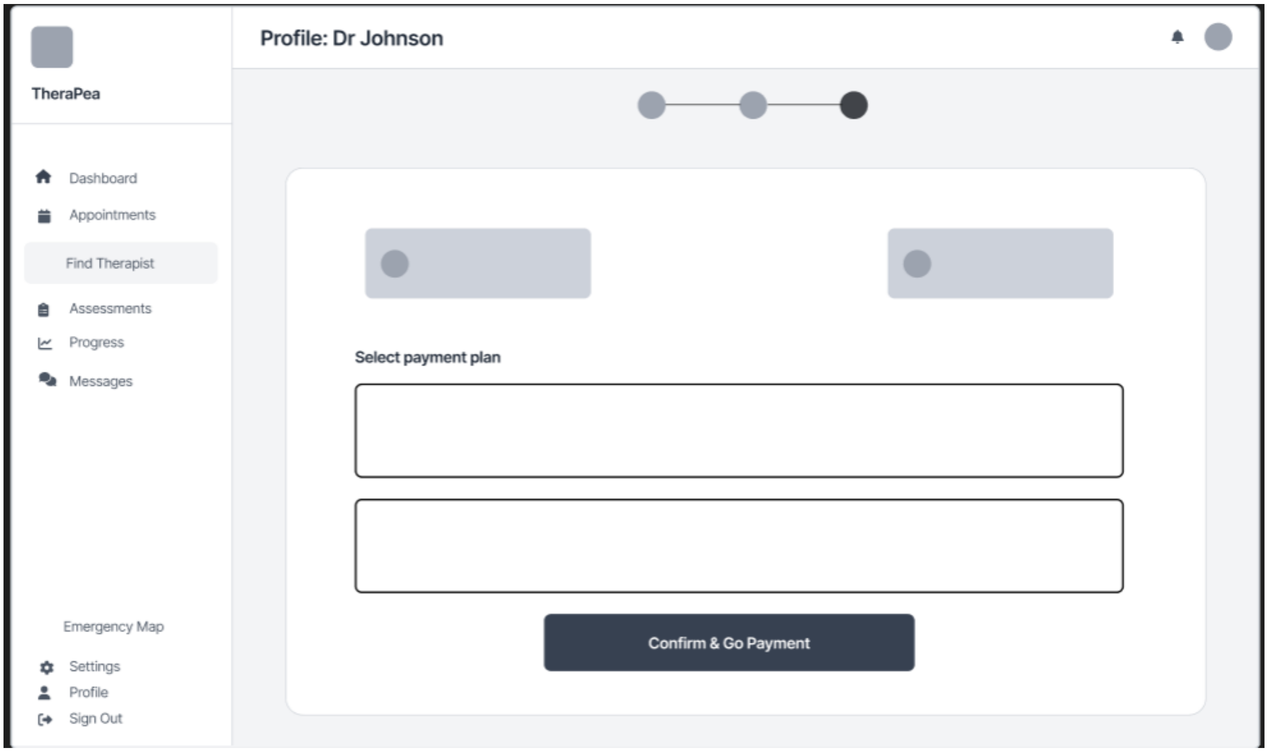
Doctor Profile & Booking:



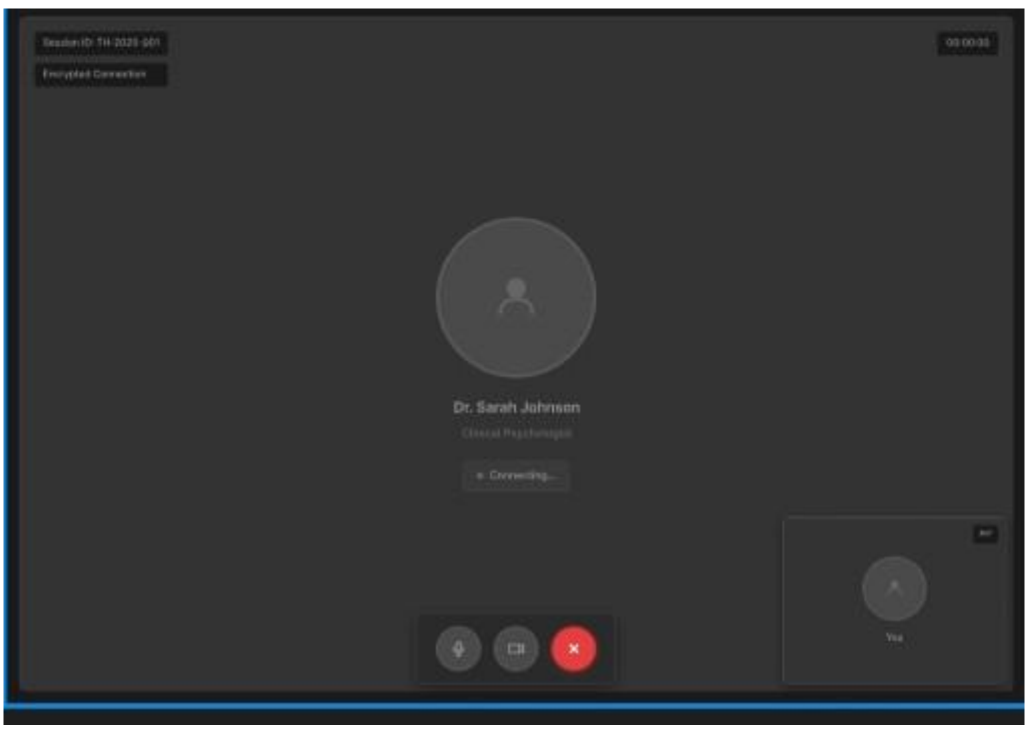
Checkout


TheraPea
Profile: Dr Johnson
Dashboard
Appointments
Find Therapist
Assessments
Progress
Messages
What brings you here?
Additional message(optional)
Continue to Scheduling
Emergency Map
Settings
Profile
Sign Out

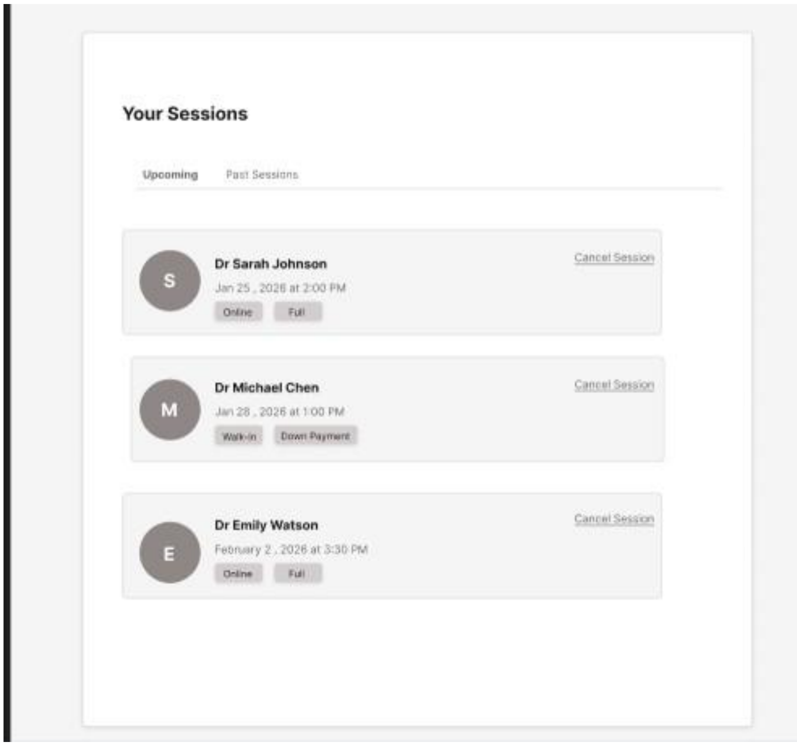

TheraPea
Profile: Dr Johnson
Dashboard
Appointments
Find Therapist
Assessments
Progress
Messages
May 2026
Su
Mo
Tu
We
Th
Fr
Sa
Available Slots
Review Summary
Emergency Map
Settings
Profile
Sign Out

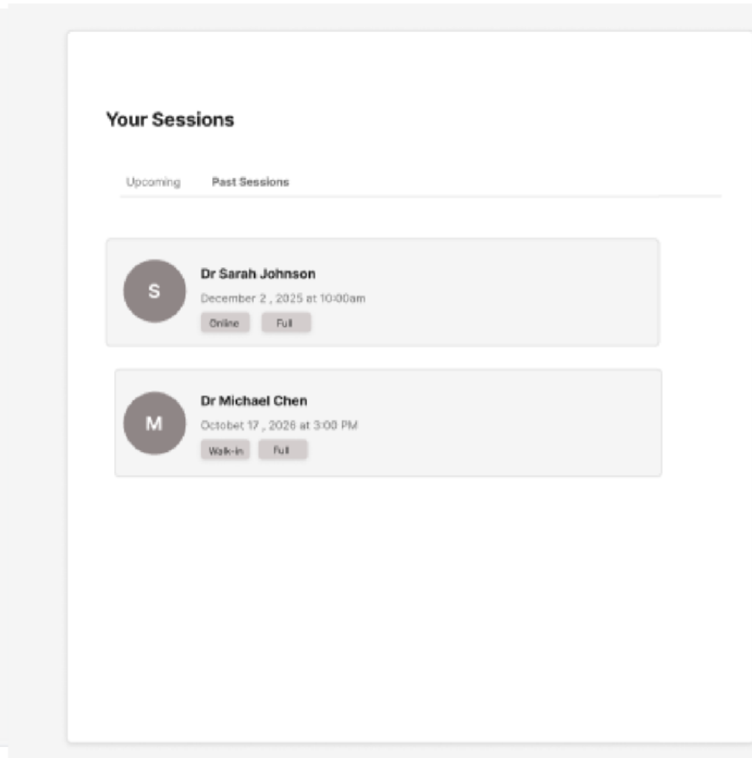


Telehealth Video Room (Agora):

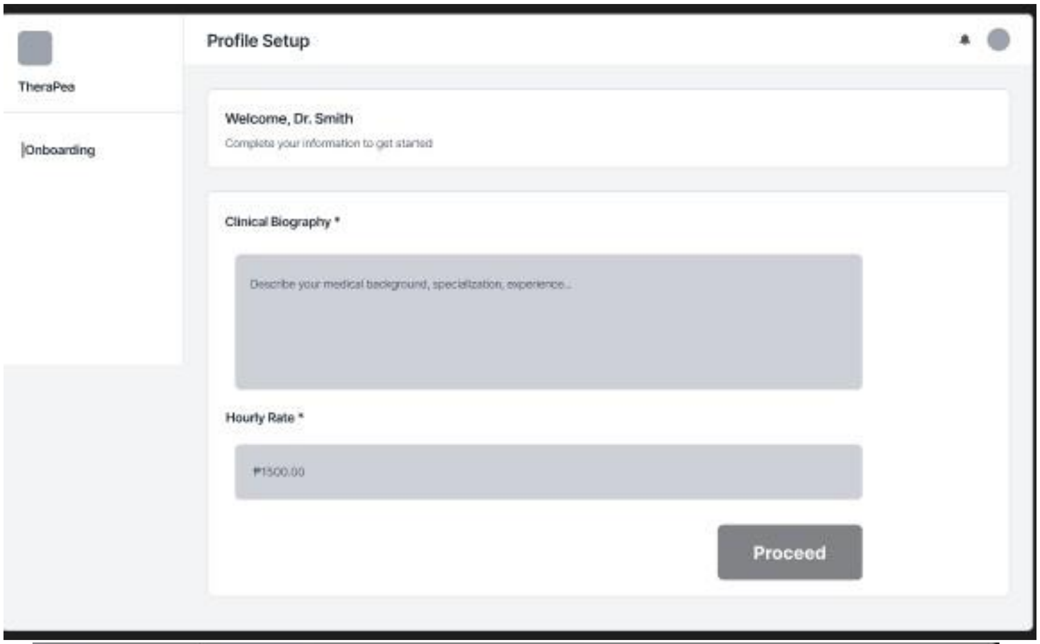


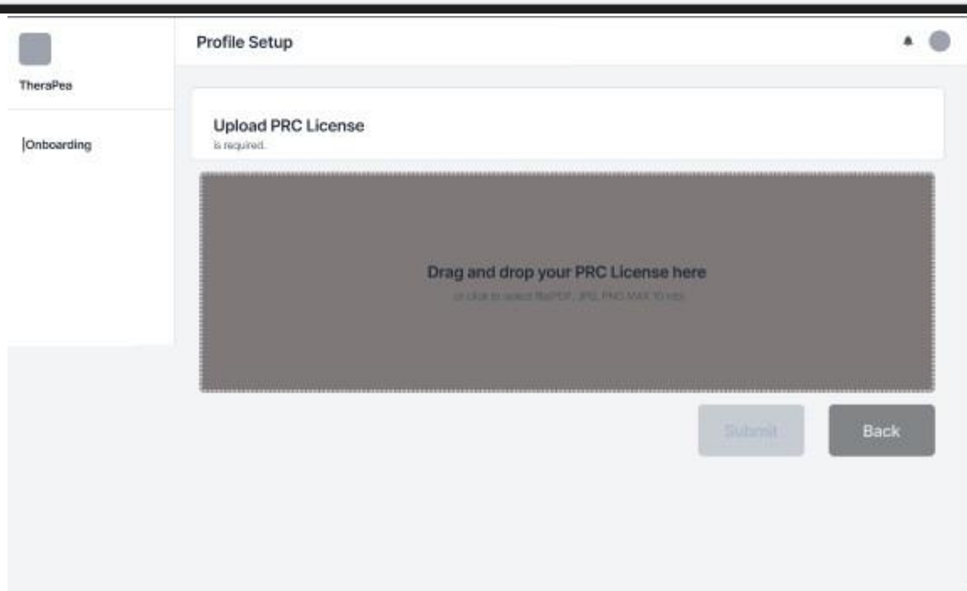
Appointments List:



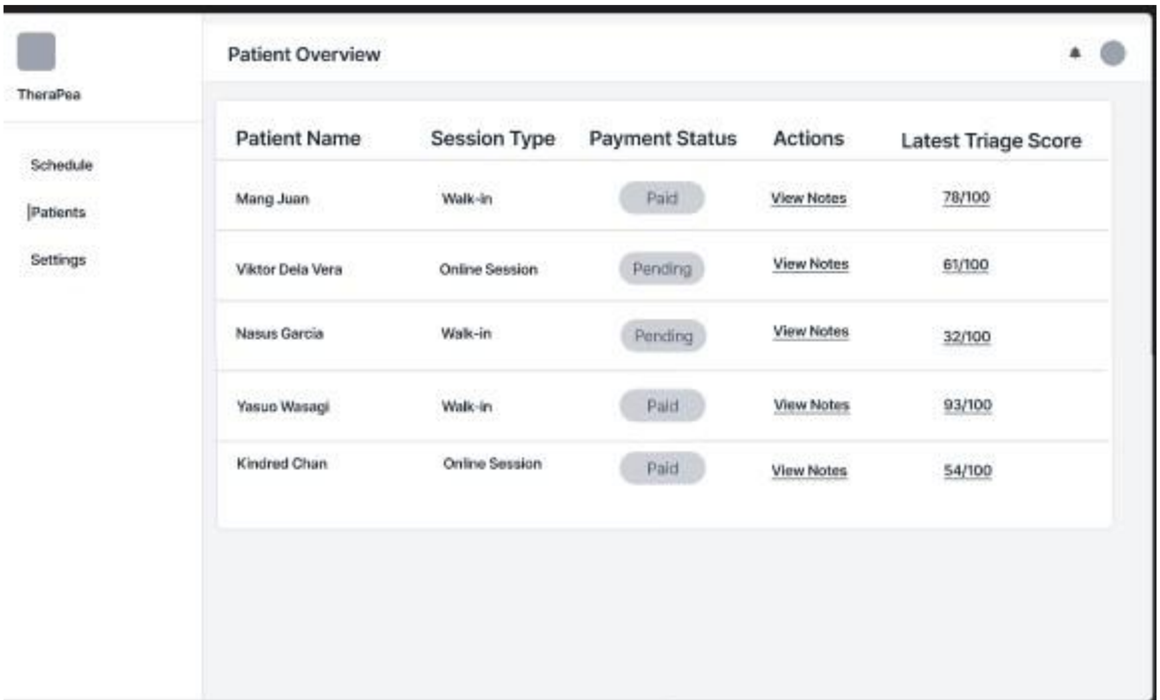


Doctor Onboarding

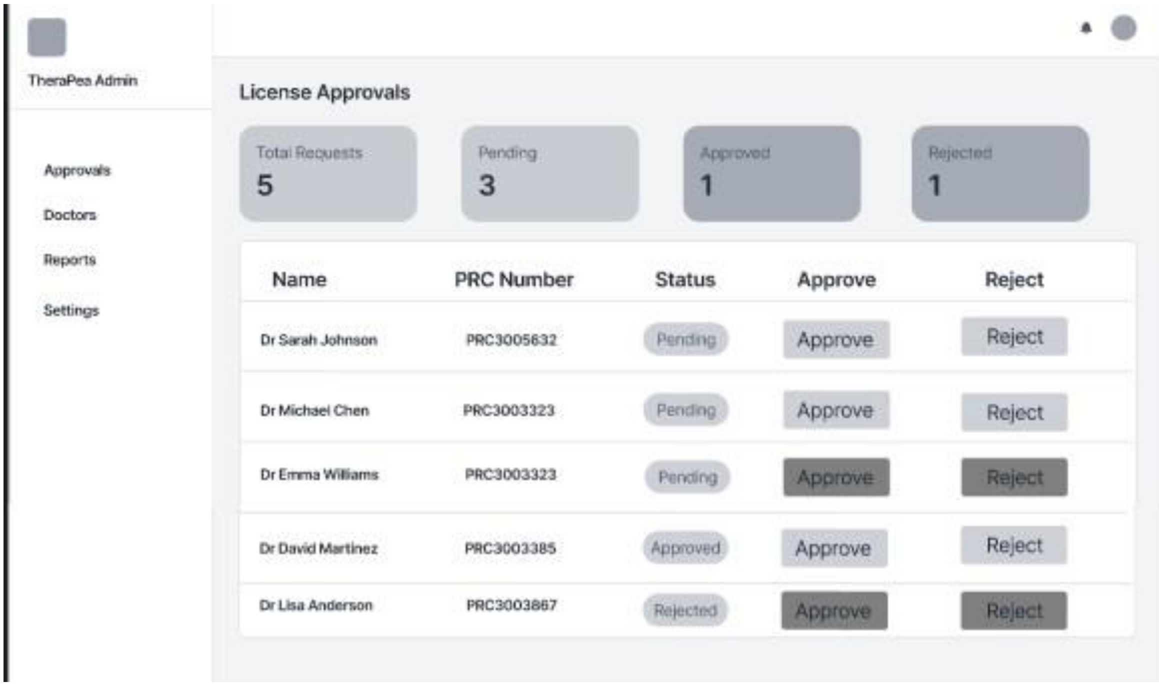

TheraPea
Onboarding
Profile Setup
Welcome, Dr. Smith
Complete your information to get started
Clinical Biography *
Describe your medical background, specialization, experience...
Hourly Rate *
₱1500.00
Proceed


TheraPea
Onboarding
Profile Setup
Upload PRC License
is required.
Drag and drop your PRC License here
Submit
Back

Patient Overview

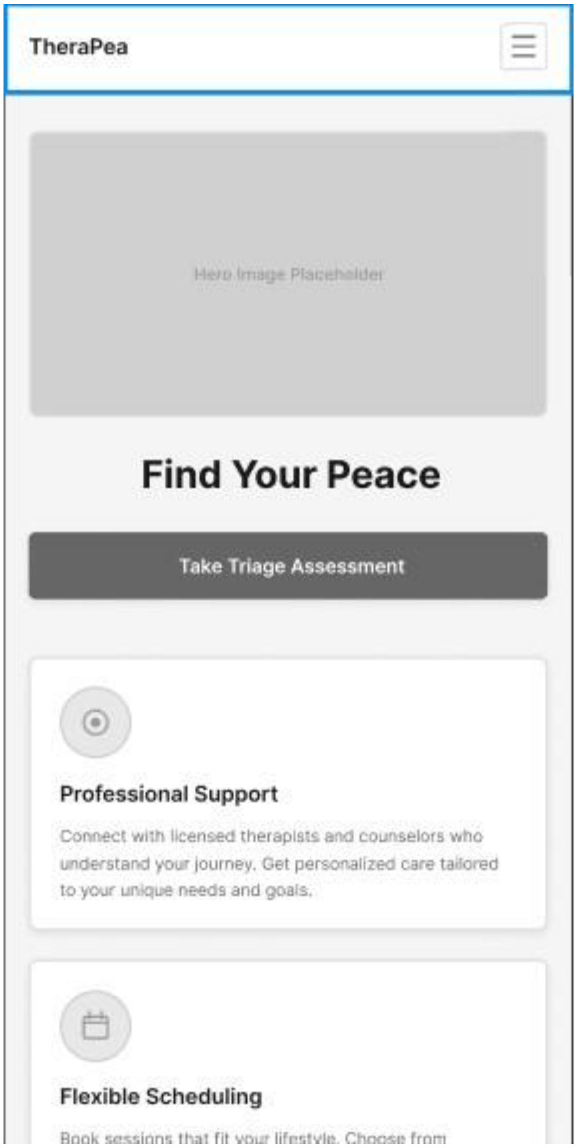


Admin Dashboard

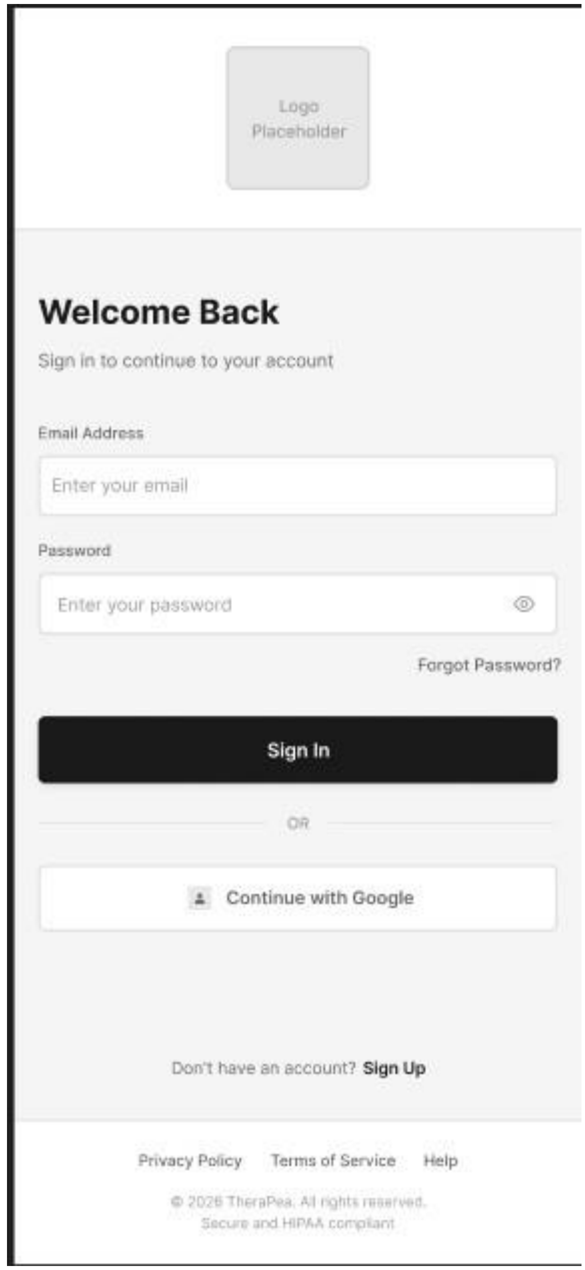


7.2 Mobile Application Wireframes

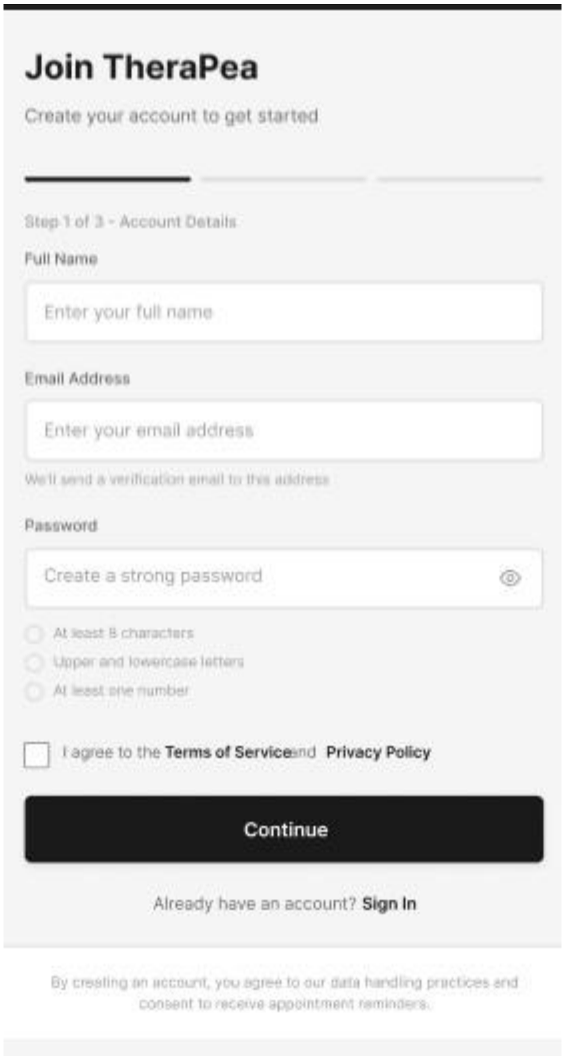
Landing page:

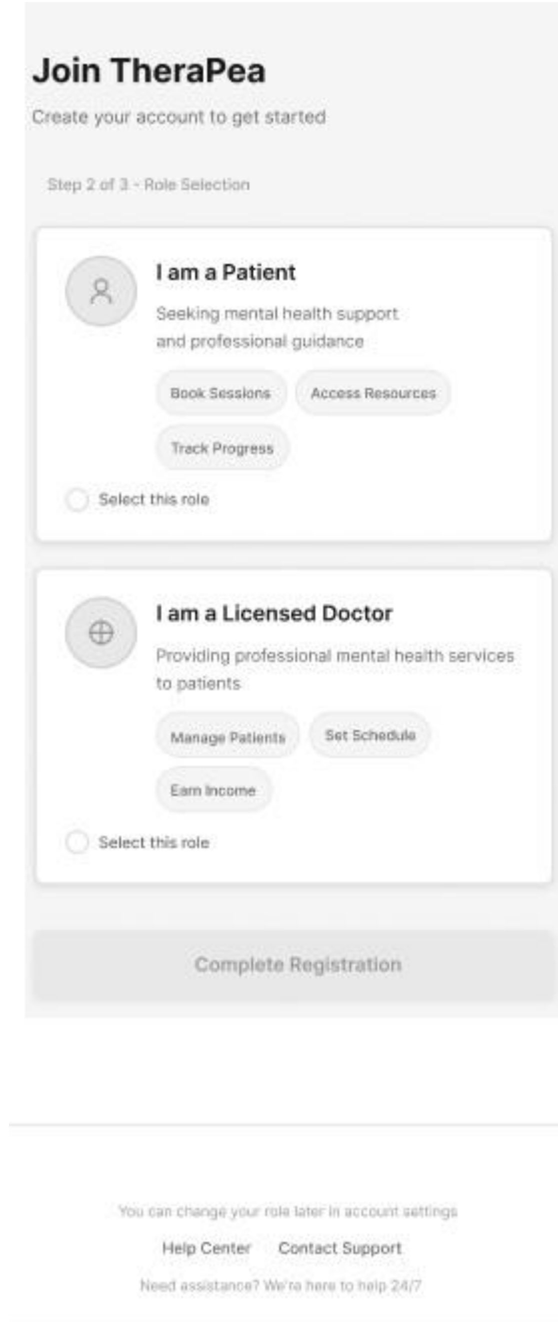


Login Page:



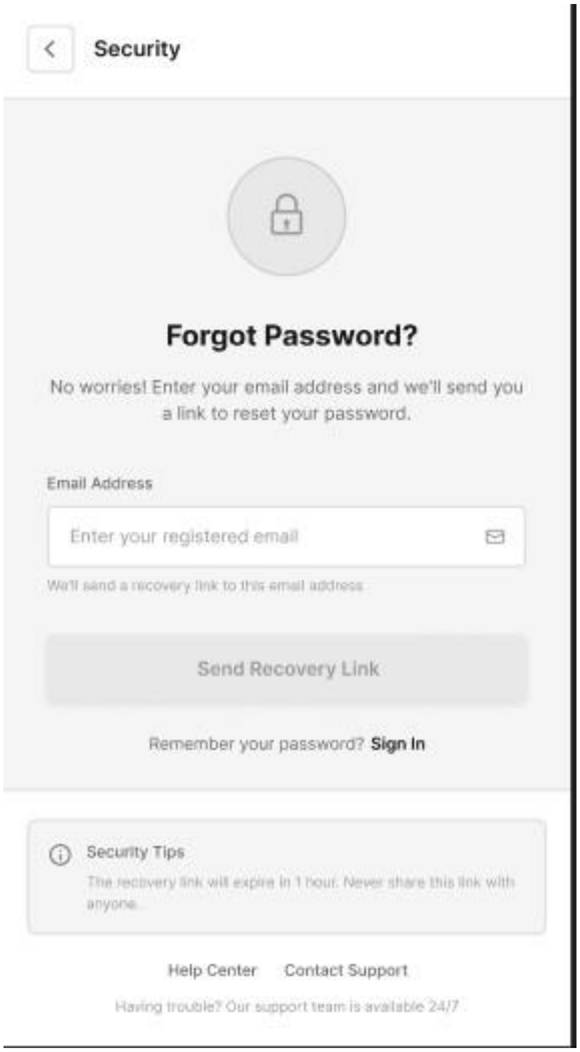
Register Page:

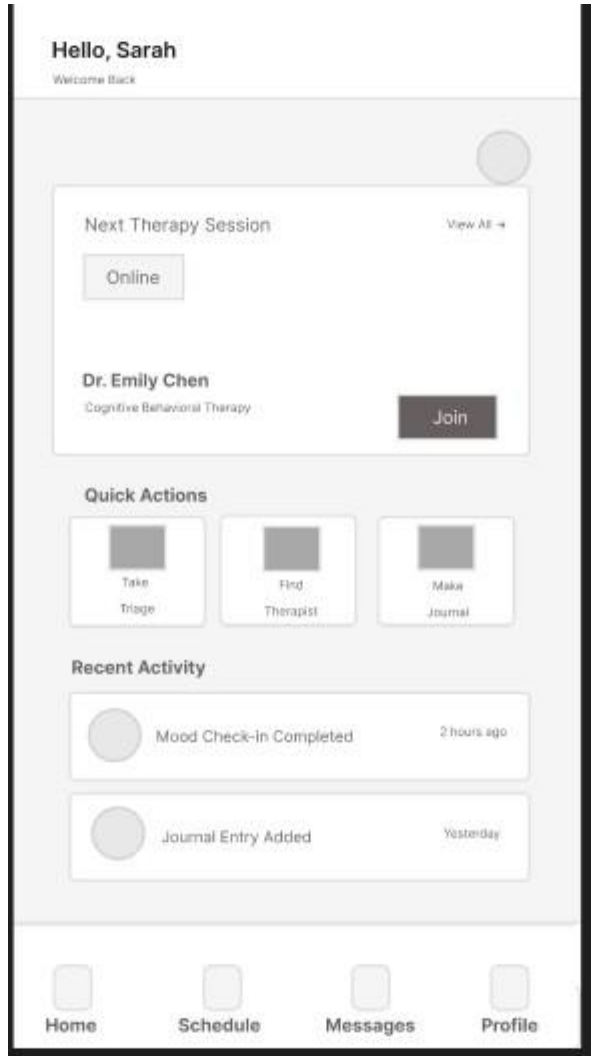




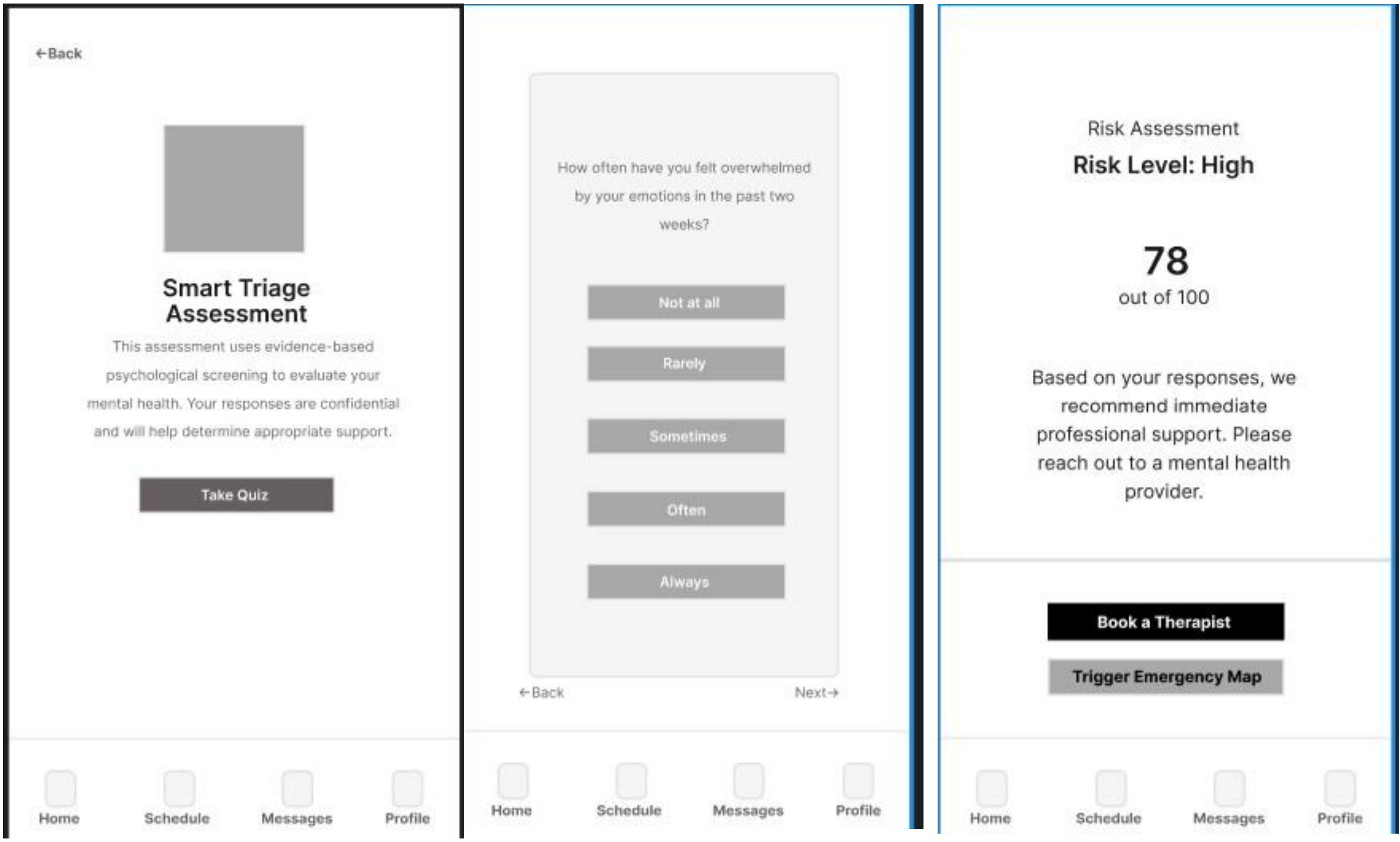
Forgot Password:

Patient Dashboard:

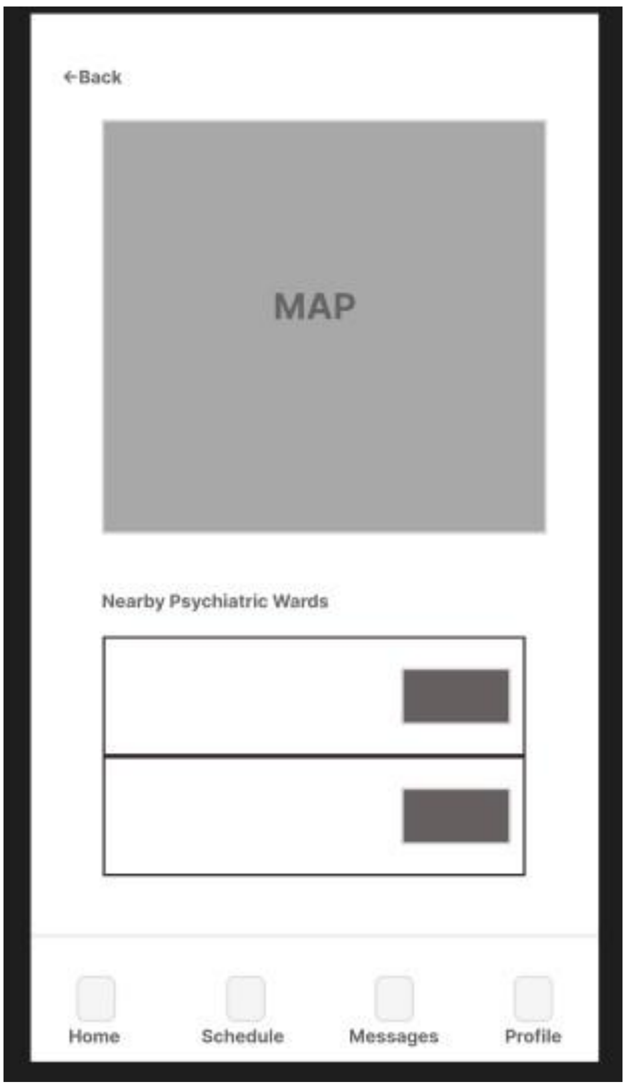




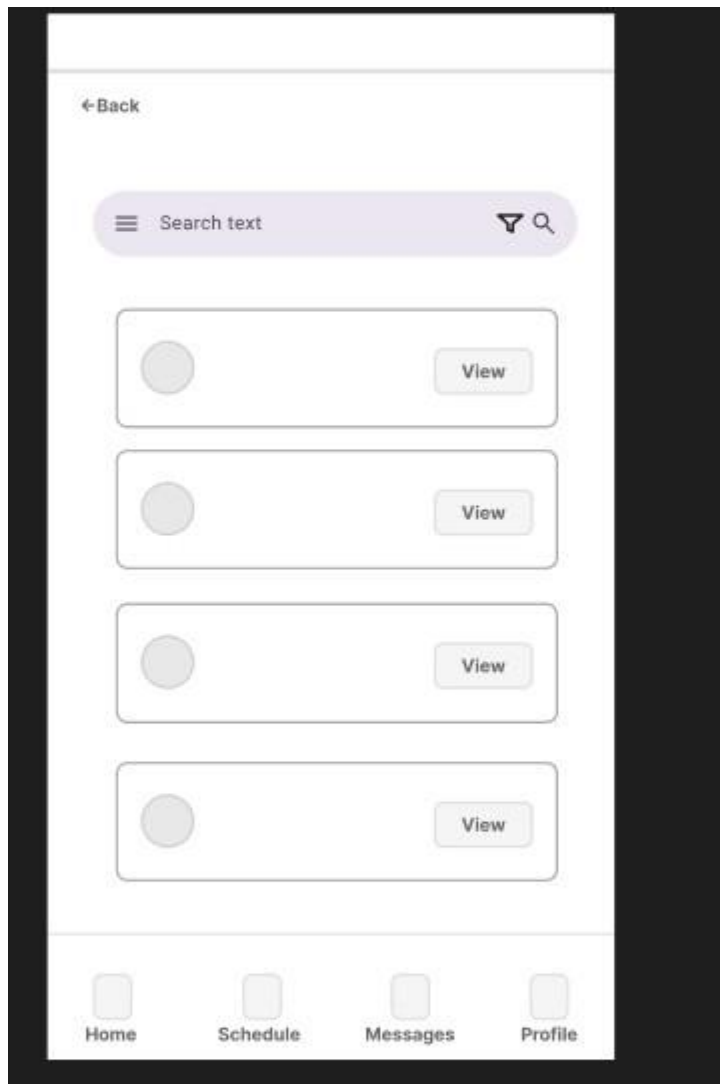
Smart Triage


←Back
Smart Triage Assessment
This assessment uses evidence-based psychological screening to evaluate your mental health. Your responses are confidential and will help determine appropriate support.
Take Quiz
Home
Schedule
Messages
Profile
How often have you felt overwhelmed by your emotions in the past two weeks?
Not at all
Rarely
Sometimes
Often
Always
←Back
Next→
Home
Schedule
Messages
Profile
Risk Assessment
Risk Level: High
78
out of 100
Based on your responses, we recommend immediate professional support. Please reach out to a mental health provider.
Book a Therapist
Trigger Emergency Map
Home
Schedule
Messages
Profile

Emergency Map:

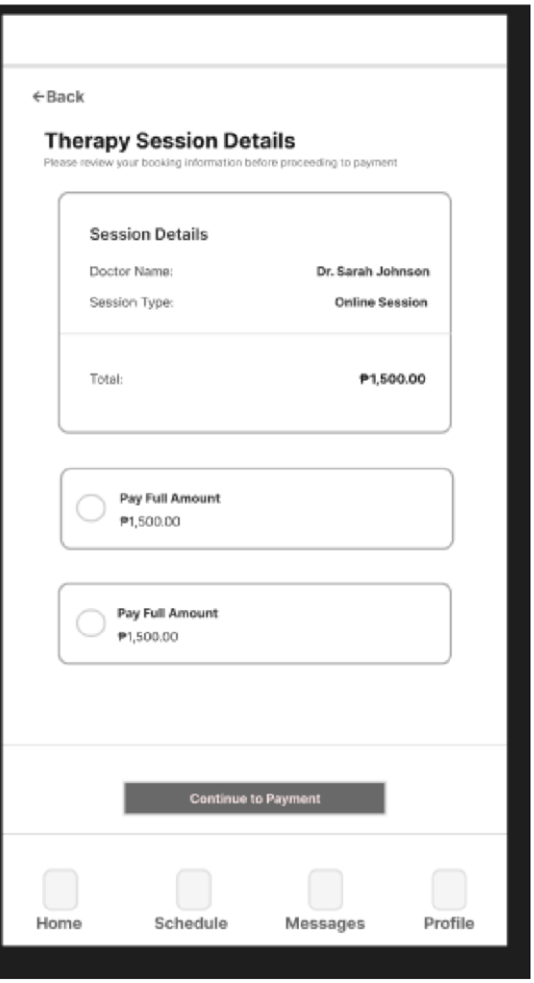


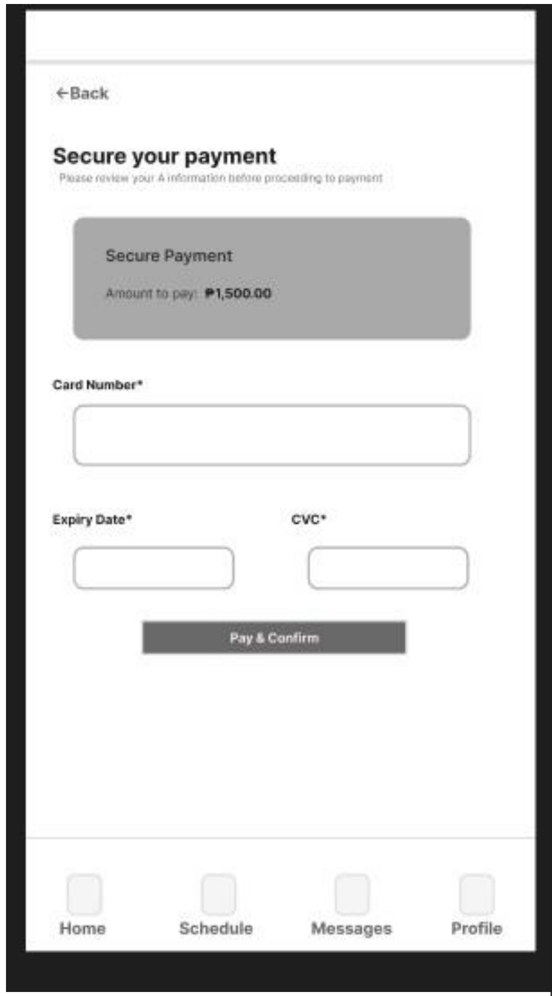
Doctor Directory:

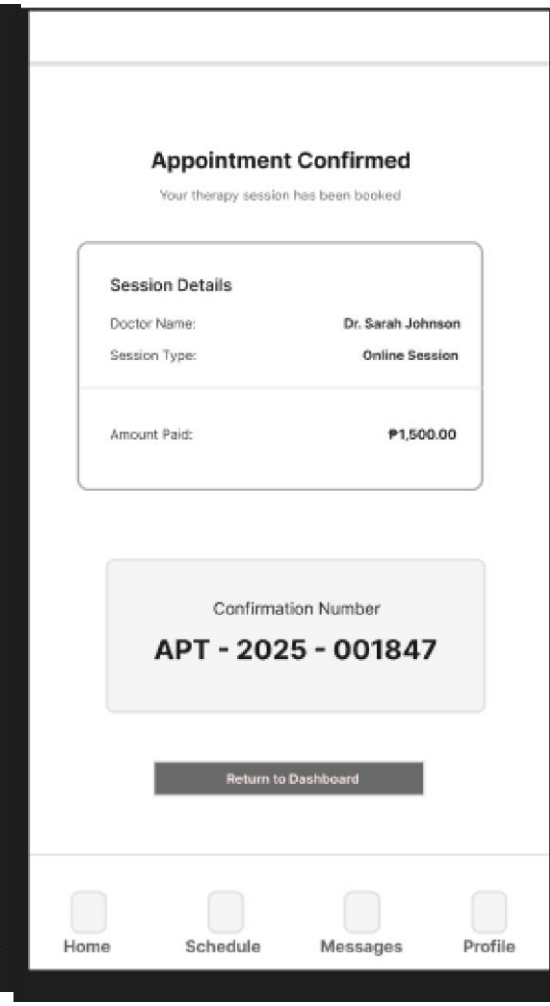


Doctor Profile & Booking:

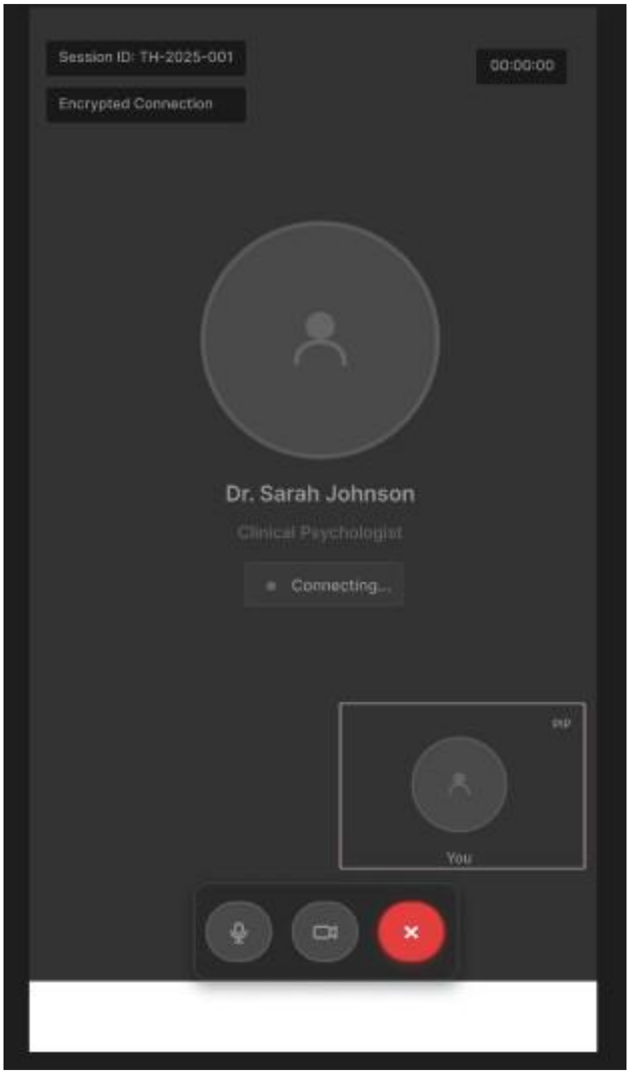
Checkout Page:



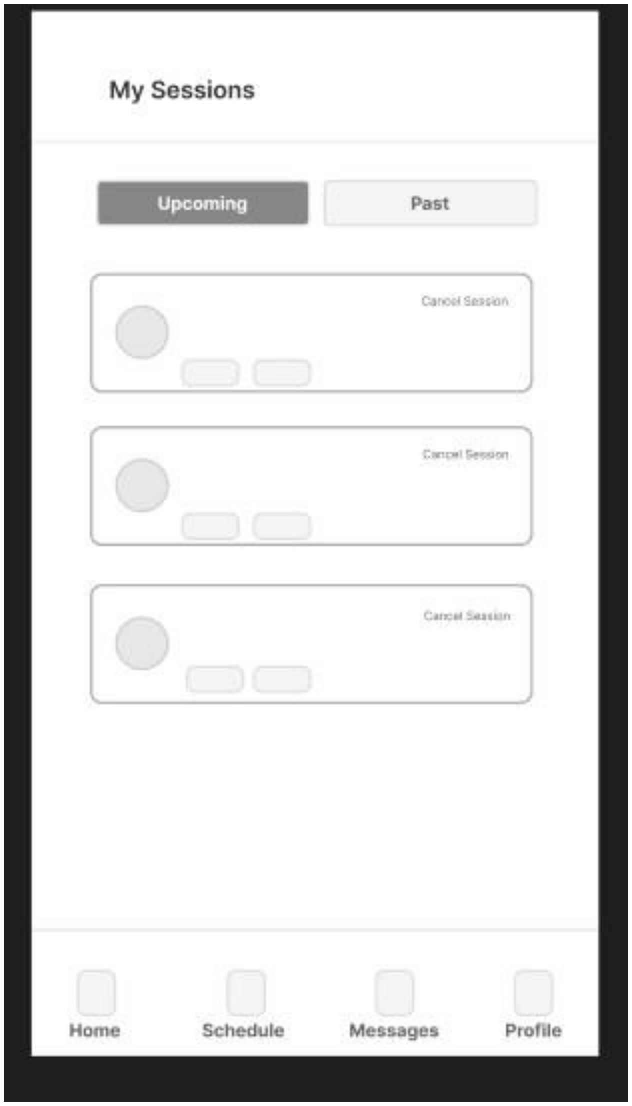


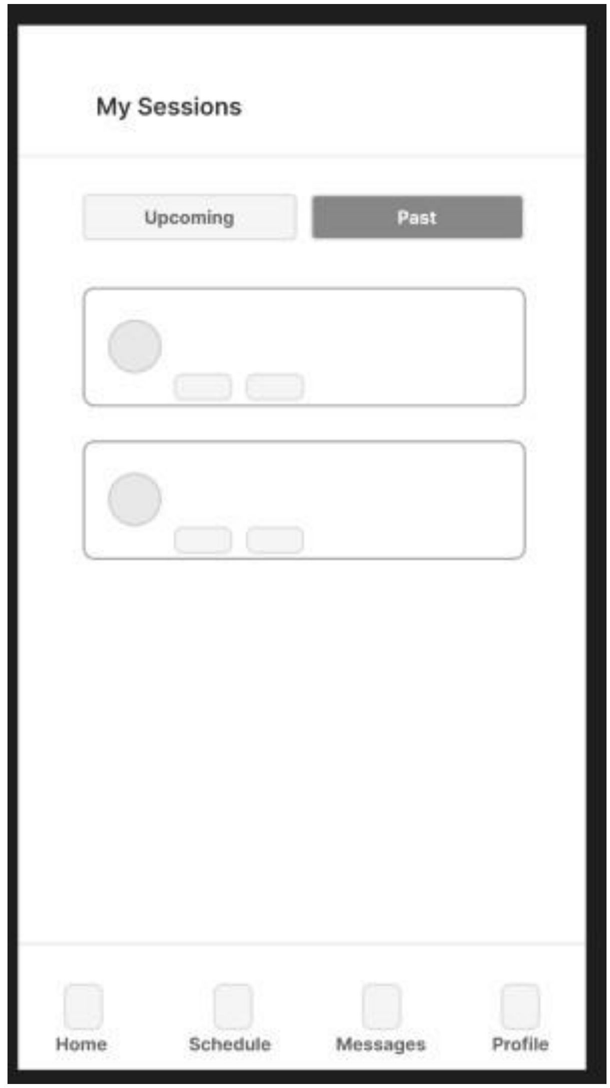


Telehealth Room Page(Agora):

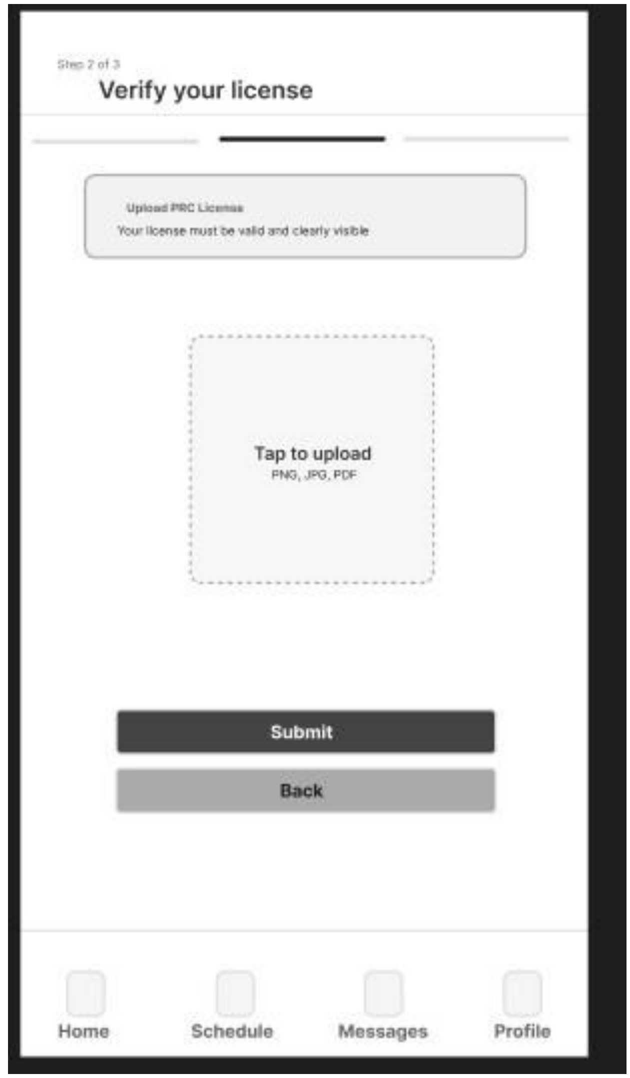


Appointments List Page:



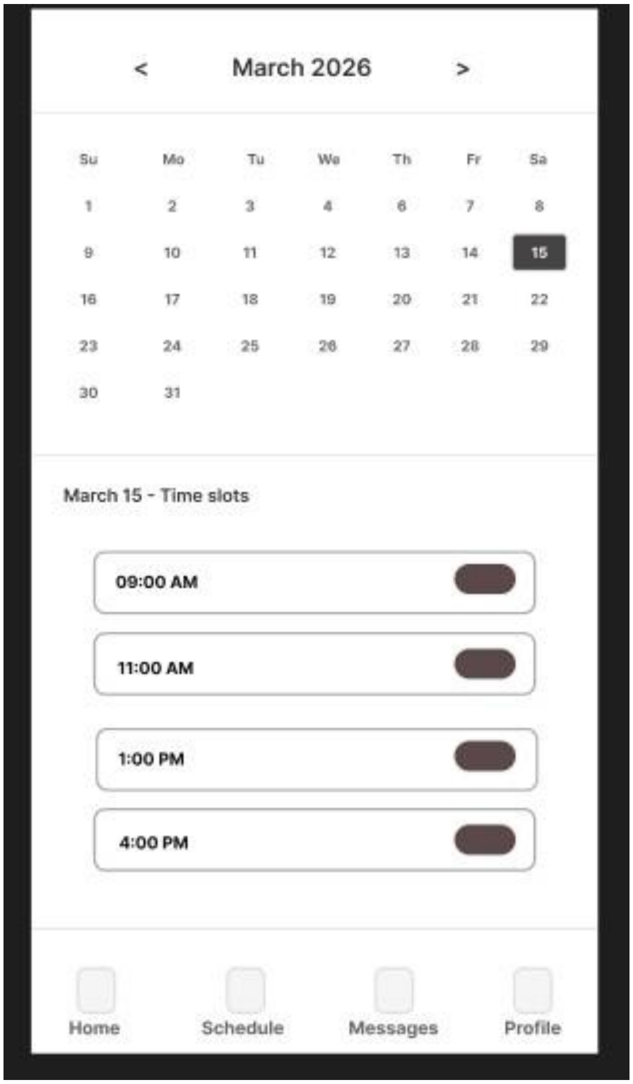


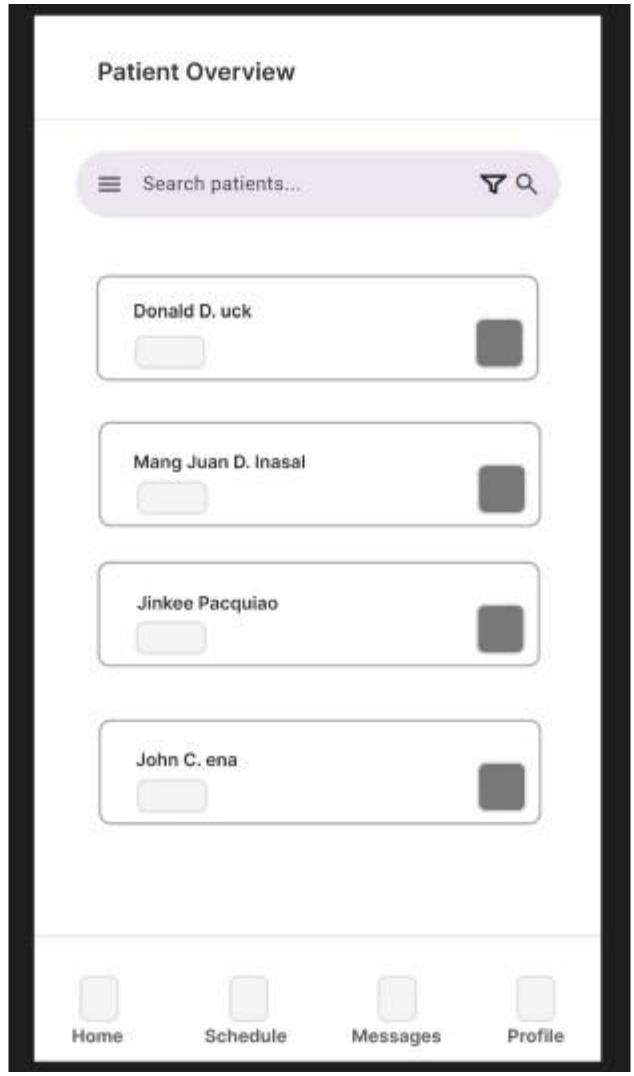
Doctors Onboarding Page:



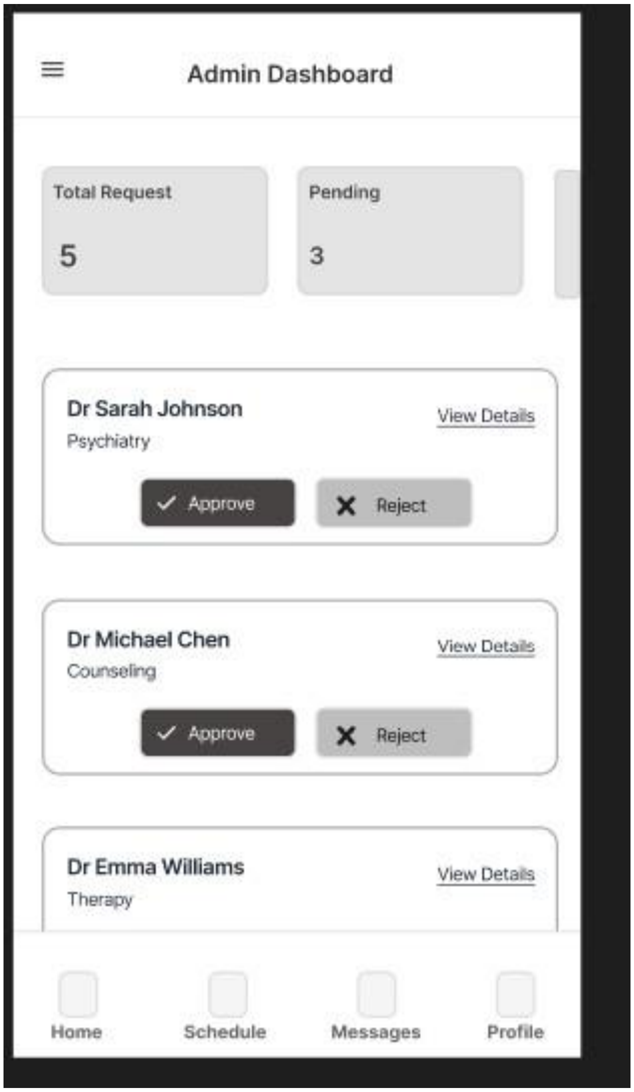
Doctor Schedule Manager Page:

Patient Overview Page:





Admin Dashboard Page:



Mobile-Specific Features:

- Crisis-Optimized Touch Targets • Low-Friction Gestures
- Offline Caching for Critical Data
- Hardware Integration
- Simplified Financial Inputs

Design System:

- **Colors (Therapeutic & Calming Palette):** ○ **Primary:** Sage Green (#8BA888) ○ **Secondary:** Soft Lavender (#A78BFA) ○ **Backgrounds:** Warm Off-White (#F9FAFB) ○ **Emergency/Alert:** Soft Coral (#F87171)

- **Typography:** Inter font family. A highly legible, rounded sans-serif font.
- **Spacing & Layout**: 8px grid system. The UI specifically uses generous margins and padding to create white space.
- **Components**: Friendly, pill-shaped buttons (fully rounded corners) to avoid harsh, sharp edges. Soft, elevated cards to group information logically.
- **Responsive Strategy:** A strict mobile-first approach for the Patient application (prioritizing accessibility and on-the-go care), while utilizing a desktop-first fluid layout (breakpoints at 640px, 768px, 1024px) for the Doctor and Admin dashboards.

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup, finalize problem statement, and objectives.

Day 3-4: Complete Functional (FRS) and Non-Functional Requirements (NFR).

Day 5-7: Finalize Layered System Architecture and Tech Stack selection.

Week 2: Detailed Design

Day 1-2: Finalize API Contracts and Endpoint Specifications.

Day 3-4: Database ERD and Schema design (Supabase/PostgreSQL).

Day 5-6: UI/UX Wireframes for Web and Mobile (Figma).

Day 7: Implementation plan finalization and repository setup.

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation & Security

Day 1: Spring Boot initialization and dependency setup.

Day 2: Supabase Database connection and Entity mapping.

Day 3: Google OAuth integration and JWT Authentication logic. Day 4: User management endpoints

Day 4: User Management and Role-Based Access Control (RBAC).

Day 5: Doctor PRC ID file upload and Admin verification endpoints.

Week 4: Core Telehealth Features

Day 1: Smart Triage Engine logic and assessment scoring endpoints.

Day 2: Appointment scheduling and availability CRUD operations. Day 3: Search and filtering

Day 3: PayMongo Payment Intent integration (Test Mode).

Day 4: Agora WebRTC token generation endpoints.

Day 5: SendGrid SMTP Email notification triggers.

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation & Patient Flow

Day 1: React 18 & TypeScript setup with Tailwind CSS.

Day 2: Authentication pages and global state management.

Day 3: Patient Dashboard and Smart Triage Questionnaire UI.

Day 4: Doctor Directory, Search, and Filtering.

Day 5: Doctor Profile and Calendar Booking component.

Week 6: Complex Integrations & Dashboards

Day 1: Checkout Flow with PayMongo embedded forms.

Day 2: Telehealth Video Room UI integration (Agora Web SDK). Day 3: Admin dashboard

Day 3: Emergency Geolocation Map (Google Maps API).

Day 4: Doctor Schedule Manager and Patient Roster.

Day 5: Admin Dashboard for PRC License approvals.

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation & Patient UI

Day 1: Android Studio setup, Kotlin, and XML-Based UI Layout architecture (API Level 34).

Day 2: Mobile Authentication and Bottom Navigation setup.

Day 3: Mobile Smart Triage swipe-interface.

Day 4: Retrofit API client setup and state management.

Day 5: Mobile Doctor Directory and Profile viewing.

Week 8: Mobile Hardware & Booking integrations Day 1:

Mobile Booking and Checkout flow.

Day 2: Mobile Emergency Map (Location Services).

Day 3: Hardware permissions (Camera/Mic) and Agora Mobile integration.

Day 4: UI polish, touch-target optimization, and offline caching.

Day 5: Testing on emulators and physical devices.

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-End testing (From Triage to Booking to Video Call).

Day 2: Webhook testing (PayMongo success triggers).

Day 3: Security review (RLS policies, JWT expiration).

Day 4: Performance testing and bug squashing.

Day 5: Documentation updates and code refactoring.

Week 10: Deployment

Day 1: Backend deployment (Railway/Render)

Day 2: Web app deployment (Vercel)

Day 3: Mobile APK generation and distribution.

Day 4: Final live-environment testing.

Day 5: Project submission **Milestones:**

- **M1 (End Week 2):** All System Design Documents complete and approved.
- **M2 (End Week 4):** Backend API, Triage Logic, and Auth fully functional.
- **M3 (End Week 6):** Web application complete (Telehealth & Payments active).
- **M4 (End Week 8):** Mobile application complete (Android APK ready).

- **M5 (End Week 10):** Full system deployed, integrated, and ready for defense.

Critical Path:

1. Authentication & Database Setup (Week 3)
2. Smart Triage Engine Logic (Week 4)
3. Appointment Booking Flow (Weeks 4 & 5)
4. Payment Gateway Integration (Week 6)
5. Telehealth Video Connection (Weeks 6 & 8)
6. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with the simplest working version of the Agora SDK and PayMongo APIs before adding custom UI elements.
- Test Webhooks and Video connections early and often using local tunneling.
- Focus strictly on the core triage-to-telehealth pipeline before attempting to add "Could Have" features like mood-tracking graphs.
- Keep automated backups of the Supabase PostgreSQL database during active development.